BCSE305L-- Embedded Systems

Dr. D. Ajitha

Module:1 Introduction 5 hours

Overview of Embedded Systems,
Design challenges,
Embedded processor technology,
Hardware Design,
Micro-controller architecture -**8051**, PIC, and ARM.

Syllabus Coverage for CAT 1:

 Module I: **Introduction**

Overview of Embedded Systems, Design challenges, embedded processor technology, Hardware Design, Micro-controller architecture -8051, PIC, and ARM.
Under Embedded Design- (GPS Moving Map and Model train examples)

 Module 2: **I/O Interfacing Techniques**

Memory interfacing, A/D, D/A, Timers, Watch-dog timer, Counters, Encoder & Decoder, UART, Sensors and actuators interfacing.
 Note: It was decided to cover Module 2 contents in connection with 8051 Microcontroller. So it is enough to cover only architecture for PIC and ARM as 8051 will be covered in detail.

Introduction to Embedded System

- System
  - Way of working, organizing, performing one or many tasks as per rules or plan
  - Arrangement – units assemble and work together as per program or plan

- Examples of system
  - Time display system – watch
  - Automatic cloth washing system – washing m/c

- Embedded Systems are
  - Omnipresent (homes, office, shopping malls, hospitals, cars, aircraft…)
  - Computing Device – does a specific focused job

What is Embedded System?

- **Embedded System**
  - any device that includes a computer but is not itself a general-purpose computer
  - h/w and s/w - part of some larger systems and expected to function without human intervention
  - respond, monitor, control external environment using sensors and actuators
- Embedding a computer - but not for general purpose
- Applied Computer System
- Includes analog interface to the external world

Examples of Embedded Systems









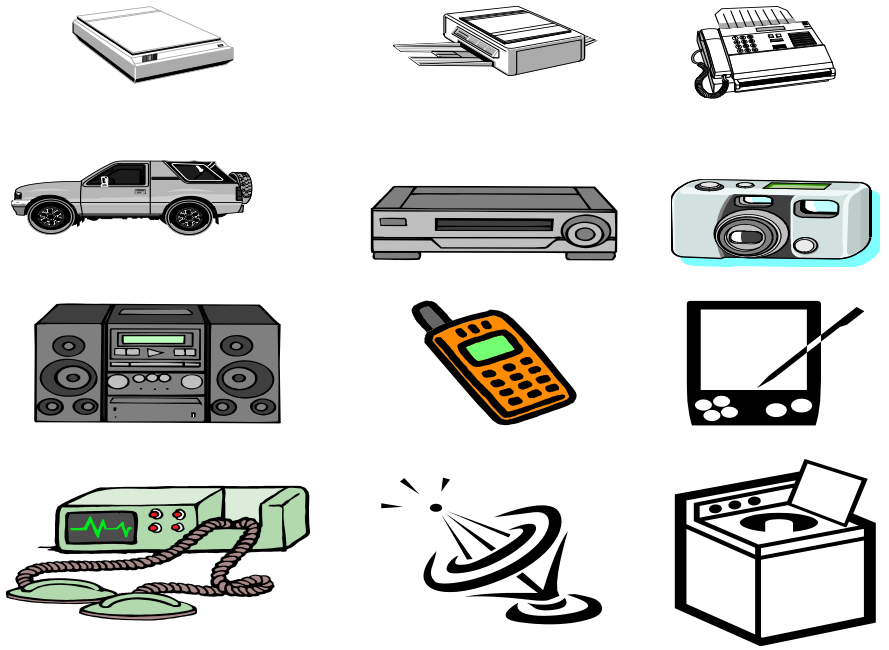





Many Different Products Depend on Embedded Systems

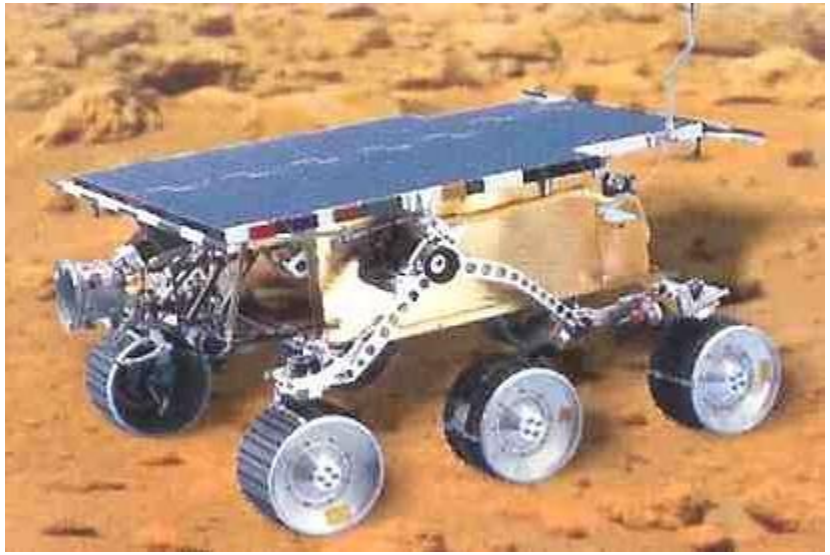
A “Short List” of Embedded Systems

Anti-lock brakes
Auto-focus cameras
Automatic teller machines
Automatic toll systems
Automatic transmission
Avionic systems
Battery chargers
Camcorders
Cell phones
Cell-phone base stations
Cordless phones
Cruise control
Curbside check-in systems
Digital cameras
Disk drives
Electronic card readers
Electronic instruments
Electronic toys/games
Factory control
Fax machines
Fingerprint identifiers
Home security systems
Life-support systems
Medical testing systems
Modems
MPEG decoders
Network cards
Network switches/routers
On-board navigation
Pagers
Photocopiers
Point-of-sale systems
Portable video games
Printers
Satellite phones
Scanners
Smart ovens/dishwashers
Speech recognizers
Stereo systems
Teleconferencing systems
Televisions
Temperature controllers
Theft tracking systems
TV set-top boxes
VCR’s, DVD players
Video game consoles
Washers and dryers



And the list goes on and on

More Examples



- NASA Mars Rover uses an Intel 80C85 8-bit microprocessor

Embedded System - Definitions

- An embedded system is an application that contains at least one programmable computer (typically in the form of a microcontroller, a microprocessor or digital signal processor chip)
  - is used by individuals who are, in the main, unaware that the system is computer-based
    - » From Embedded C Programming perspective by Michael.J.Pont

- "An embedded system is a system that has software embedded into computer-hardware, which makes a system dedicated for an application (s) or specific part of an application or product or part of a larger system."
  - s/w usually embeds into a ROM or flash
  - Independent system or part of a large system
    - » by Raj Kamal

Embedded System - Definitions

- “An embedded system is one that has a dedicated purpose software embedded in a computer hardware.”
  - » By Raj Kamal

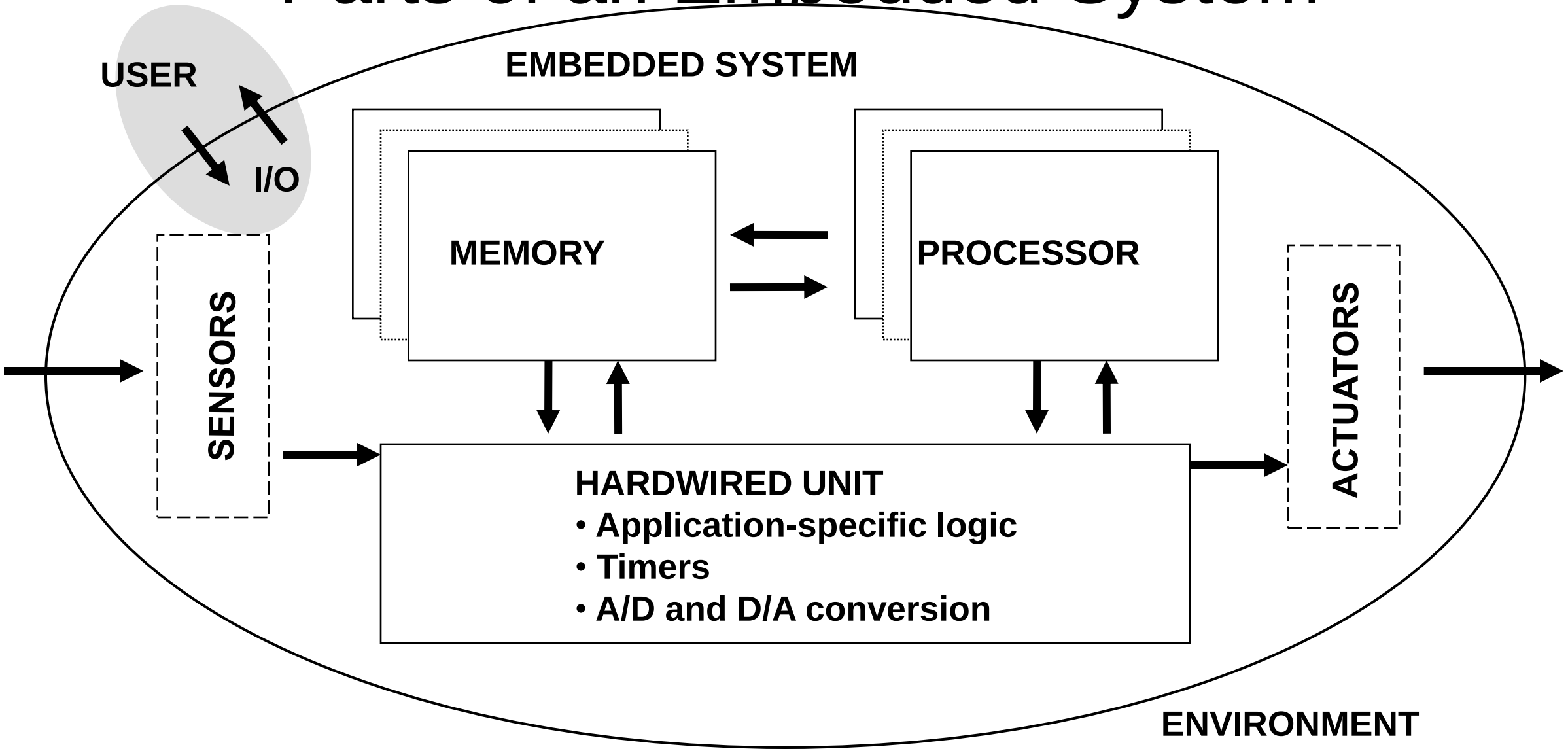
- Any device that includes programmable computer but not self intend to be a general purpose computer
  - » Wayne Wolf

- Electronic system contains microcontroller or microprocessor but not general purpose computers
  - computer is hidden or embedded in the system
    - » Todd D. Morton

- Combination of Software and Hardware in which the software controls the entire hardware for a dedicated application
  - » Raj Kamal

- A general-purpose definition of embedded systems is that they are devices used to control, monitor or assist the operation of equipment, machinery or plant. “Embedded” reflects the fact that they are an integral part of the system. In many cases, their “embeddedness” may be such that their presence is far from obvious to the casual observer**.**
  - >> **Institute of Electrical Engineers (IEE)**

Parts of an Embedded System



Parts of an Embedded System (cont.)

- Actuators - mechanical components (e.g., valve)
- Sensors - input data (e.g., accelerometer for airbag control)
- Data conversion, storage, processing
- Decision-making

- Range of implementation options
- Single-chip implementation: system on a chip

Embedded System Vs Desktop System

- Desktop / Laptop
  - General purpose computer
  - Used for playing games, word processing, accounting, SDT etc.,

- Embedded System
  - Single Purpose and
  - fixed embedded software for specific job

- Typical Examples
  - A/C, VCD/DVD Player, Printer, Fax m/c, Mobile phone etc
  - Customized embedded hw + fixed embedded sw (firmware) + specific processor
    - to meet the specific requirement

Definition

- Embedded computing system: any device that includes a programmable computer but is not itself a general-purpose computer.
- Take advantage of application characteristics to optimize the design:
  - don’t need all the general-purpose bells and whistles.

Regular VS Embedded Software Development

Regular Software Development:

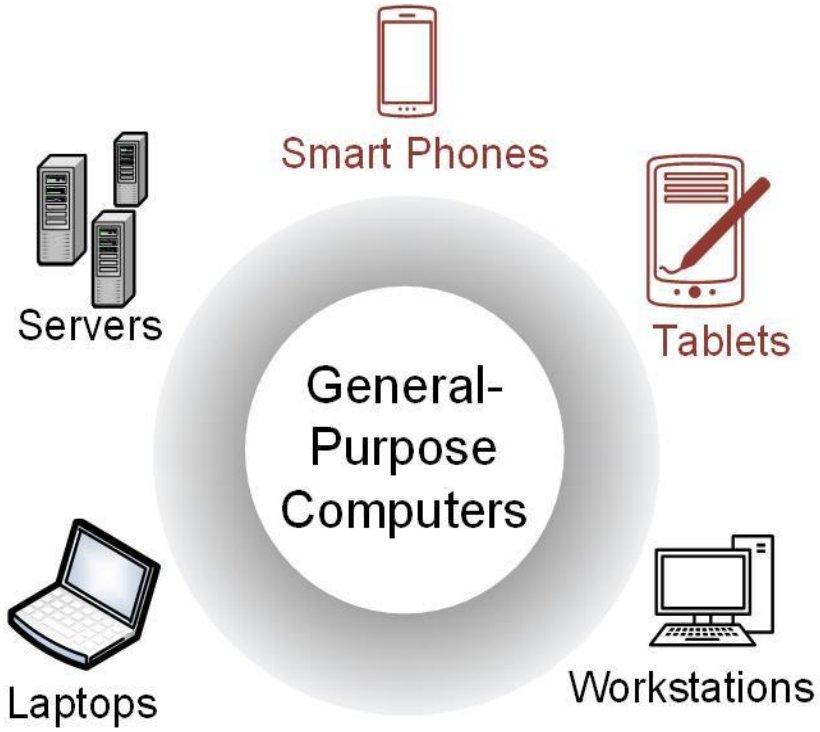
- Client-Server Architecture
- Operating System Compatibility
- Platform Independent
- User Interface Design
- Rapid Application Development
- Standardized Development Frameworks
- Scalable Architecture
- Widely Used Technologies
- Web Development Technologies
- Wide Range of Development Tools

Embedded Software Development:

- Specific Hardware Platform
- Real-Time Constraints
- Firmware Development
- Memory Management
- Efficient Power Consumption
- Low-Level Programming Languages
- Hardware-Software Integration
- Performance Optimization
- Customized Development Process
- Minimalistic User Interface

General-Purpose Computers

- Able to run a variety of software.
- Contain relatively high-performance hardware components (fast processors, data & program storage).
- Require an operating system (OS).

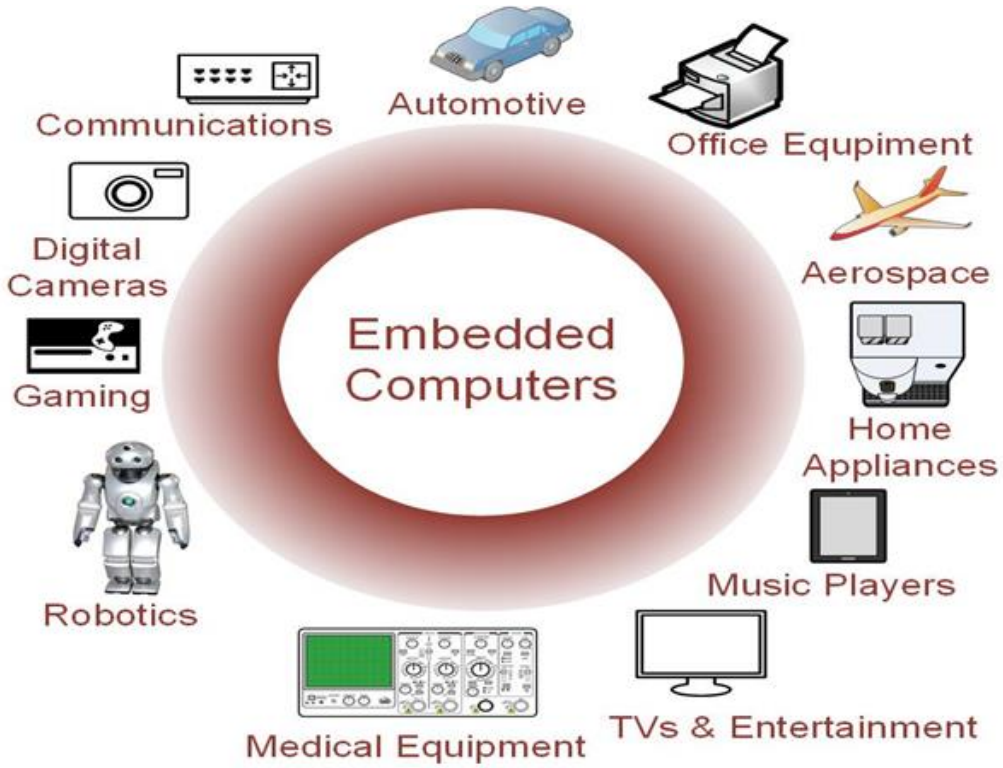


General-Purpose Computers

- Designed for heavy user interaction.
- Uses a variety of peripherals (displays, keyboards, mouse, internet connections, wireless communication capability).
- Expensive ($100s - $1000s).
- Use a group of integrated circuits or chips (ICs).
  - One implements the central processing unit (CPU).
- Several implement data memory and program storage.

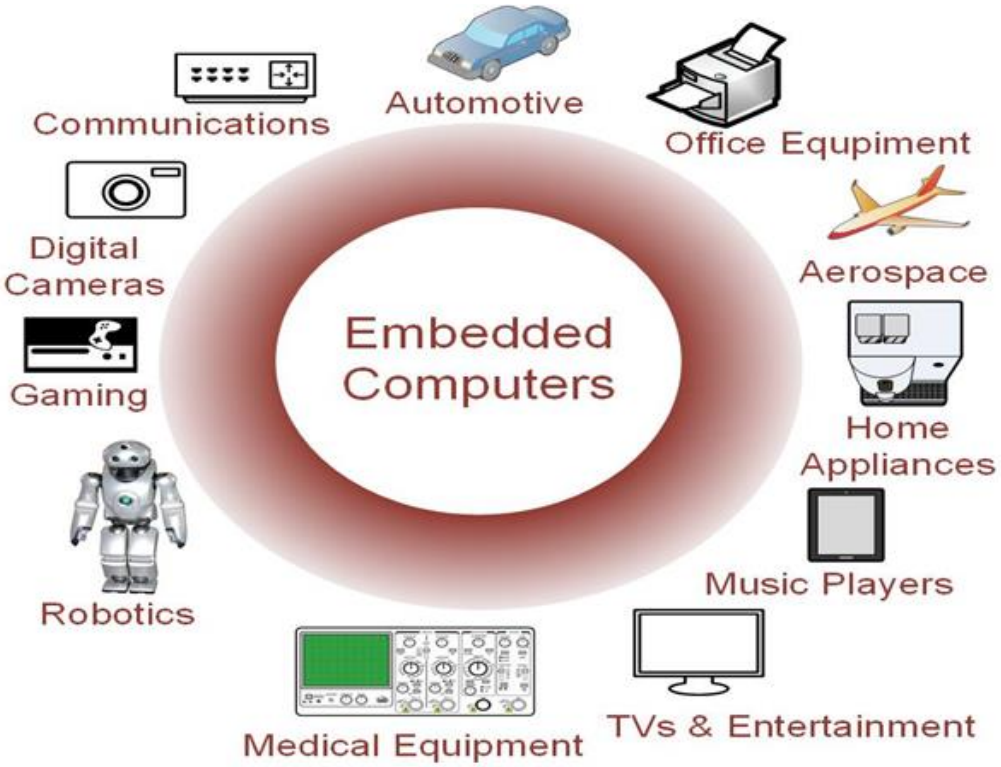
EMBEDDED COMPUTERS

- Resources can be implemented on a single IC.
- Include a variety of peripherals (timers, analog-to-digital converters, digital-to-analog converters, serial interfaces).
- Small size makes them very versatile.



Embedded Computers

- Contains firmware (only the *needed* software which is not intended to be changed frequently).
- May contain Real Time Operating Systems (RTOS) which are used as a task scheduler.
- Low cost (10s of cents to a few dollars).



Embedded Computer Attributes

- Embedded in the system they control.
- Enough performance to "get the job done".
- Implemented on a single chip.
- Software not intended to be changed by user.
- Low-cost (pennies to dollars).

Embedded Computers

Communications

Automotive

Office Equipment

Digital Cameras

Aerospace

Gaming

Home Appliances

Robotics

Music Players

Medical Equipment

TVs & Entertainment

Note: Smart phones and tablets were originally considered embedded systems. But in the past decade they have become so sophisticated that they have become general-purpose.

Smart Phones

Tablets

Servers

General-Purpose Computers

Laptops

Workstations

General-Purpose Computer Attributes

- Abundant hardware resources to support running many types of software.
- Sufficient performance to run many types of software.
- Sophisticated operating systems to manage resources (Windows, iOS, Linux).
- Usually implemented across many chips (CPS vs. RAM vs. Hard Drive).
- Higher-cost (100's to $1,000's).

Embedded Systems Software Development Tools

MPLAB

Arduino Software

PSpice

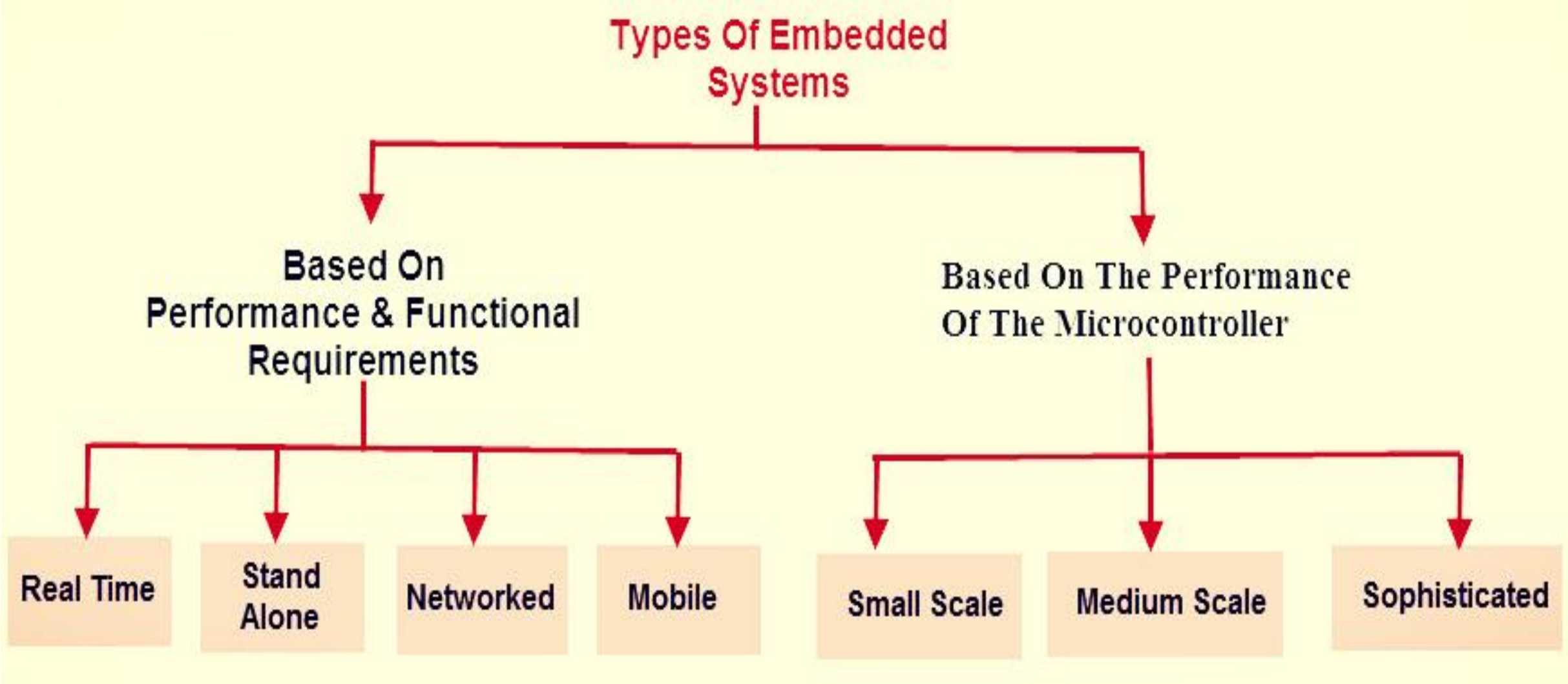
MATLAB

Proteus

Visual Studio

LabVIEW

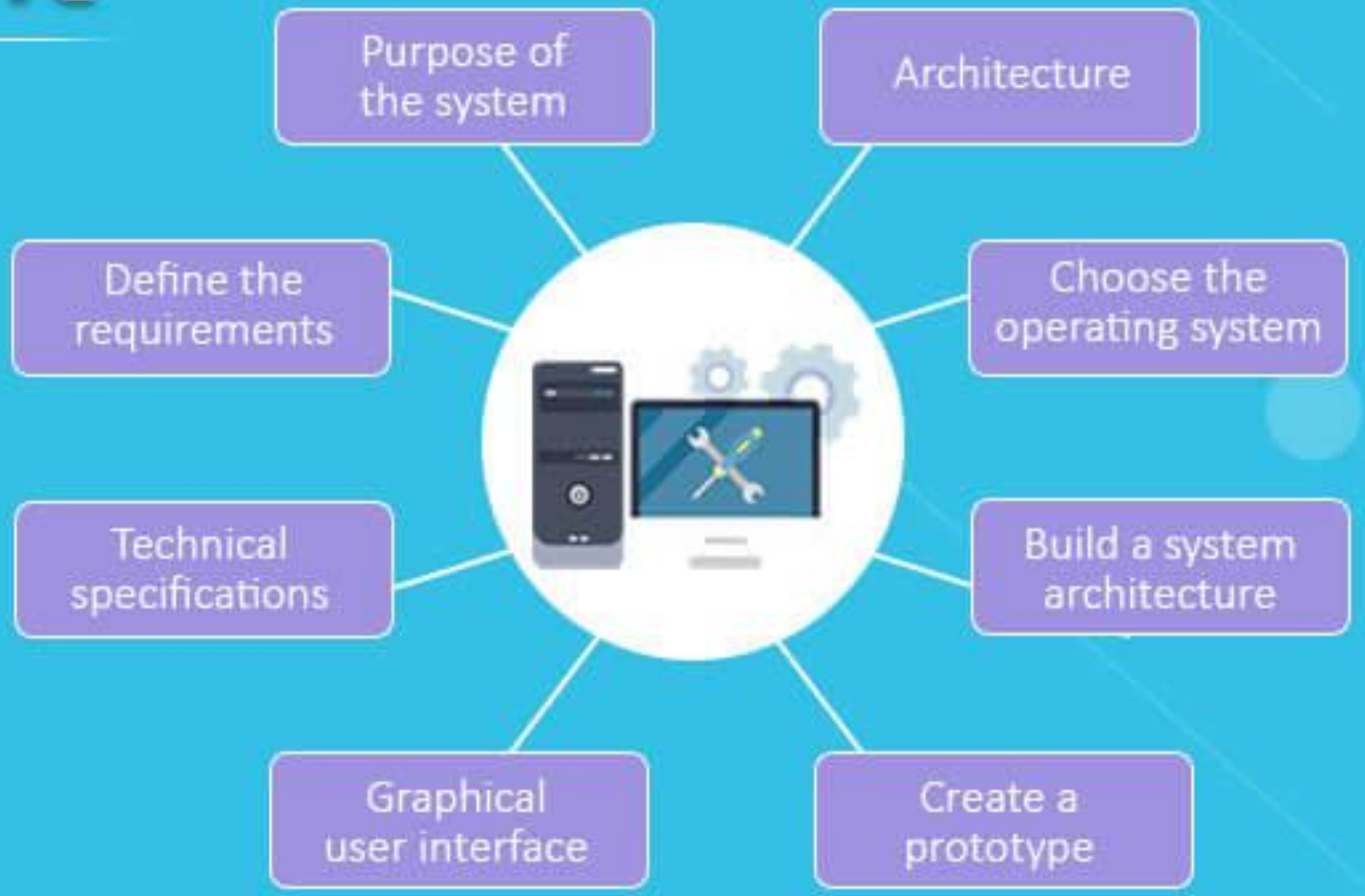
Keil

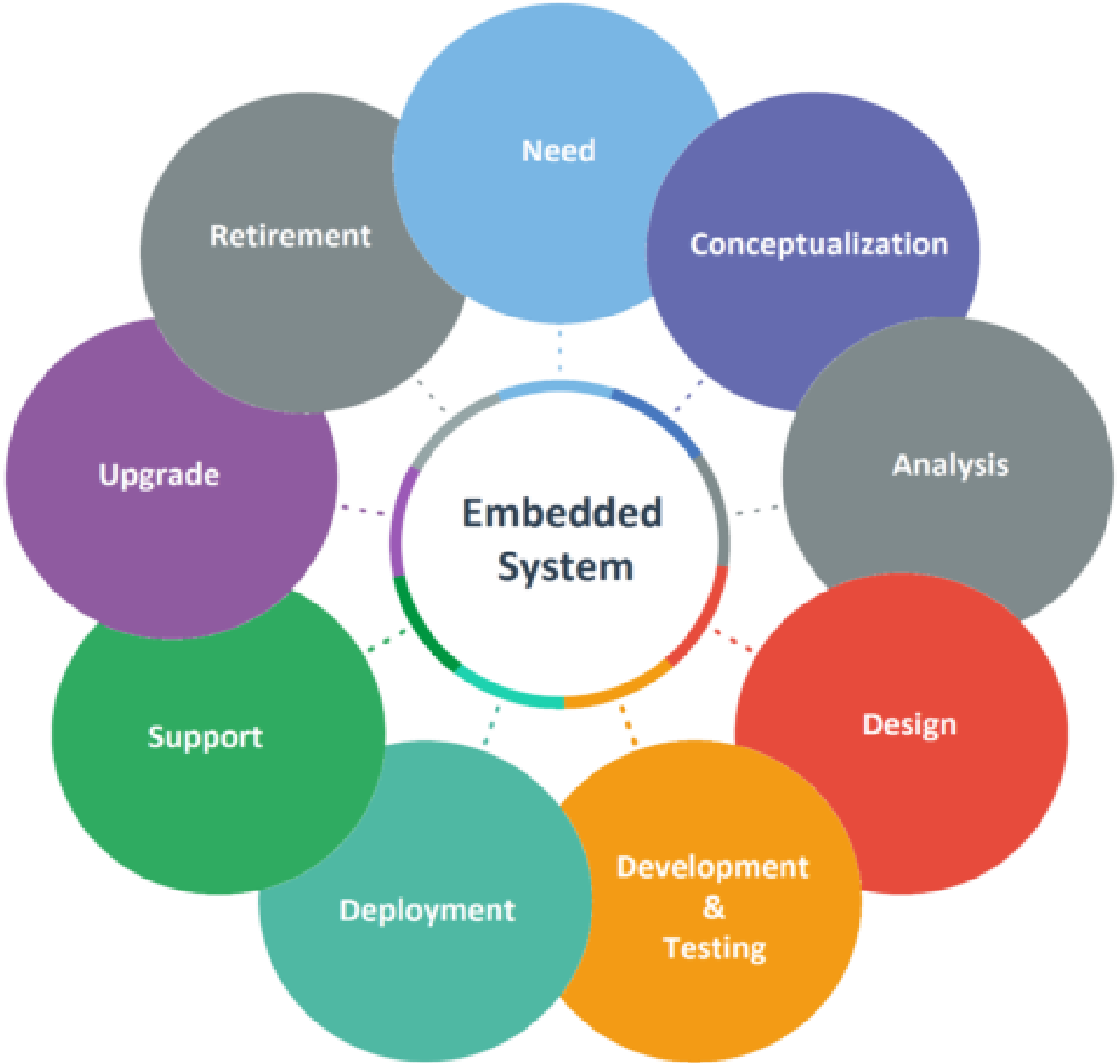


https://www.geeksforgeeks.org/classification-of-embedded-systems/

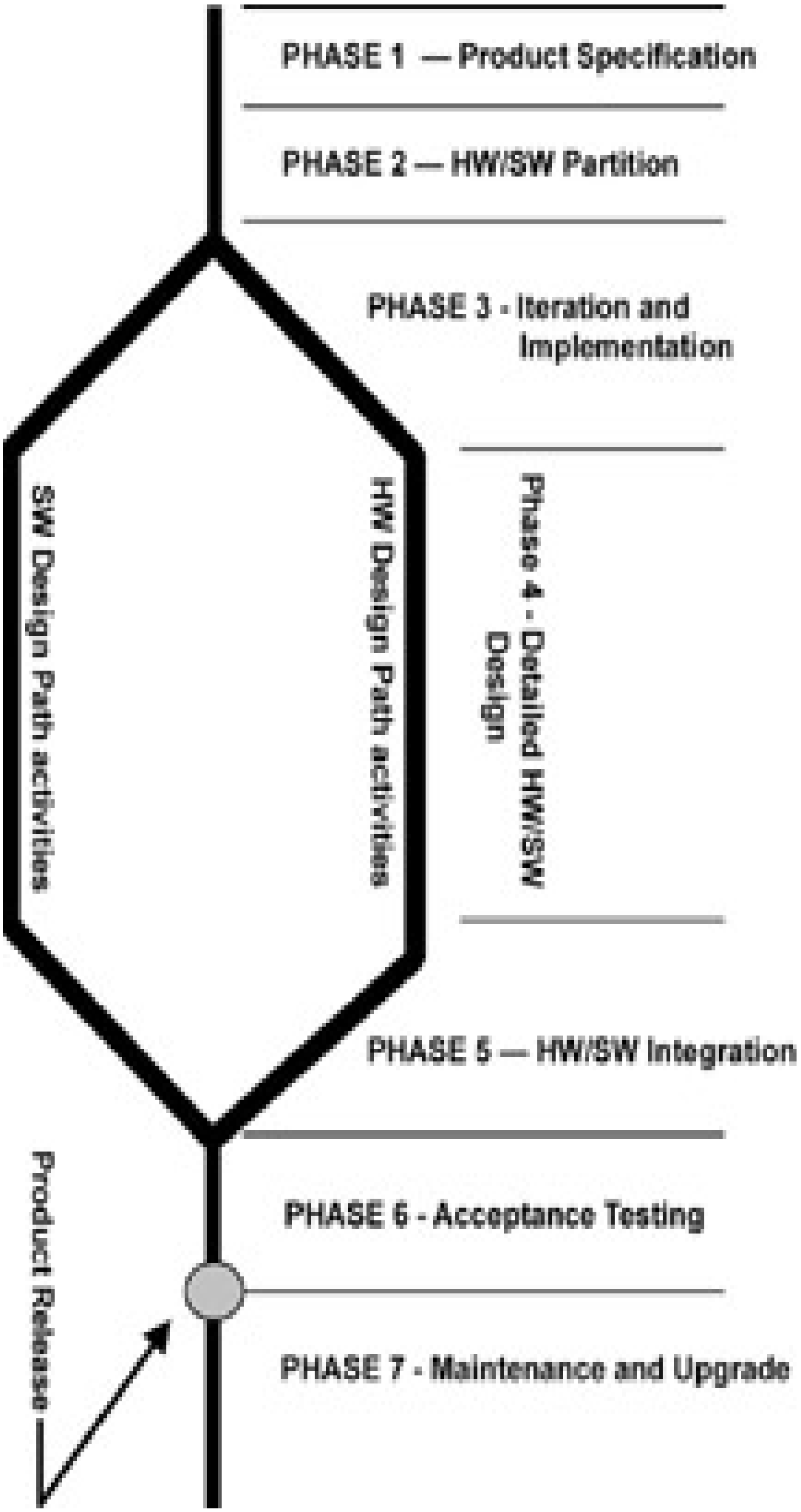
https://unstop.com/blog/classification-of-embedded-systems

Embedded System Development




Need
Conceptualization
Analysis
Design
Development & Testing
Deployment
Support
Upgrade
Retirement
Embedded System


4
IMPLEMENTATION
5
TESTING & INTEGRATION
3
DESIGN
Development Process of Embedded Systems
6
MAINTENANCE
2
ANALYSIS
1
PLANNING


PHASE 1 — Product Specification
PHASE 2 — HW/SW Partition
PHASE 3 - Iteration and Implementation
Phase 4 - Detailed HW/SW Design
HW Design Path activities
SW Design Path activities
PHASE 5 — HW/SW Integration
PHASE 6 - Acceptance Testing
PHASE 7 - Maintenance and Upgrade
Product Release

Design methodologies

- A procedure for designing a system.
- Understanding your methodology helps you ensure you didn’t skip anything.
- Compilers, software engineering tools, computer-aided design (CAD) tools, etc., can be used to:
  - help automate methodology steps;
  - keep track of the methodology itself.

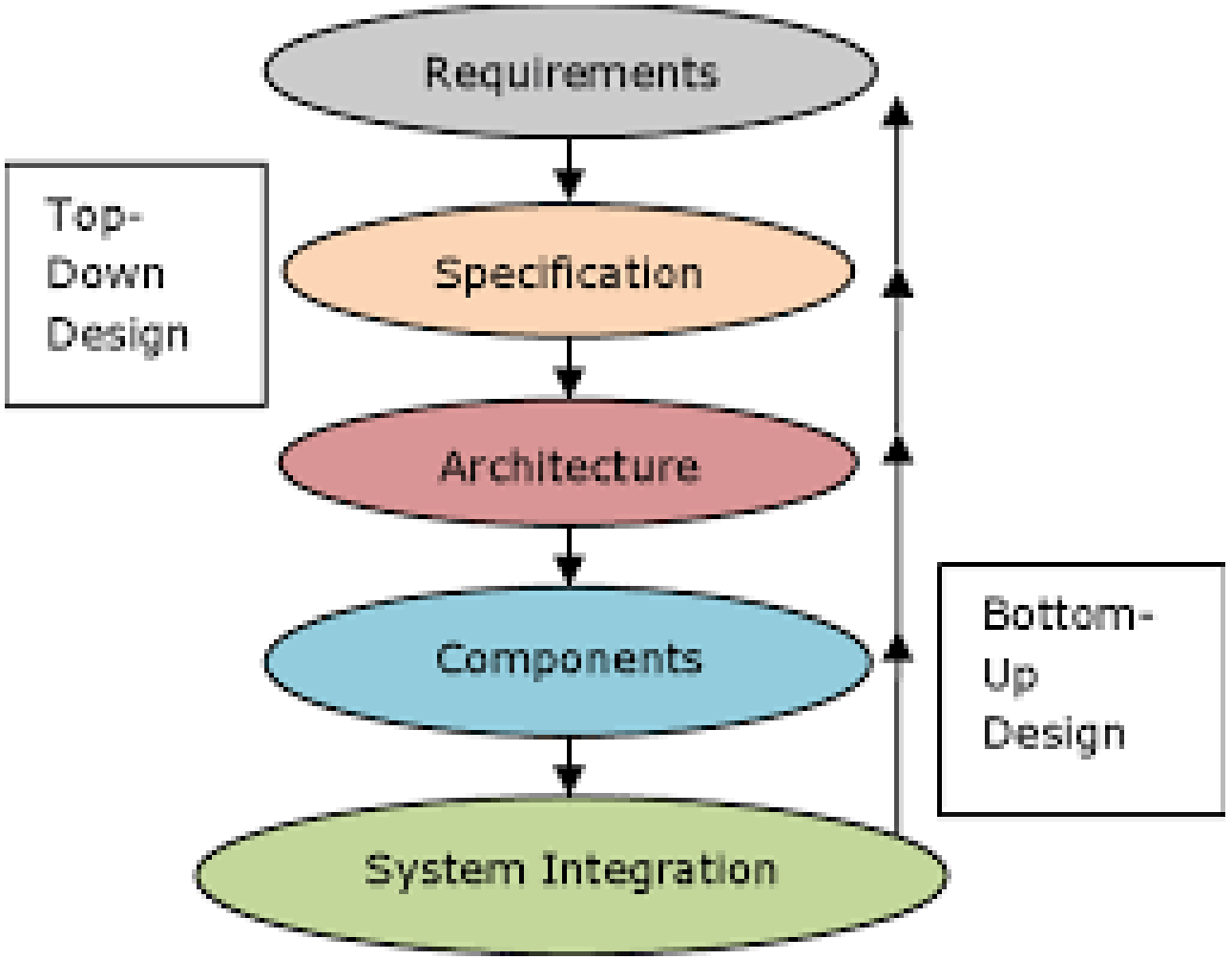
ES Design Goals

- **Performance-** In real-time systems, performance means meeting deadlines. Missing the deadline by even a little is bad. Finishing ahead of the deadline may not help.
  - overall speed, deadlines

Functional vs. non-functional requirements

- Functional requirements:---Working (functions) of the System
  - output as a function of input.
- Non-functional requirements:----Quality of the System
  - time required to compute output;-**Performance-speed**
  - **size, weight**, etc.;
  - **power consumption;-critical in battery powered devices-**cost increases
  - **reliability;**
  - Many embedded systems are mass-market items that must have low manufacturing **costs.** Limited memory, microprocessor power, etc.
  - etc.

Embedded System Design Process



Top-down vs. bottom-up

- Top-down design:
  - start from most abstract description;
  - work to most detailed.
- Bottom-up design:
  - work from small components to big system.
- Real design uses both techniques.

Stepwise refinement

- At each level of abstraction, we must:
  - analyze the design to determine characteristics of the current state of the design;
  - refine the design to add detail.
  - verify the design to ensure that it meets all goals

Requirements

- Plain language description of what the user wants and expects to get.

- May be developed in several ways:

  - talking directly to customers;

  - talking to marketing representatives;

  - providing prototypes to users for comment.

Designing hardware and software componentS

- Must spend time architecting the system before you start coding.
- Some components are ready-made, some can be modified from existing designs, others must be designed from scratch.

System integration

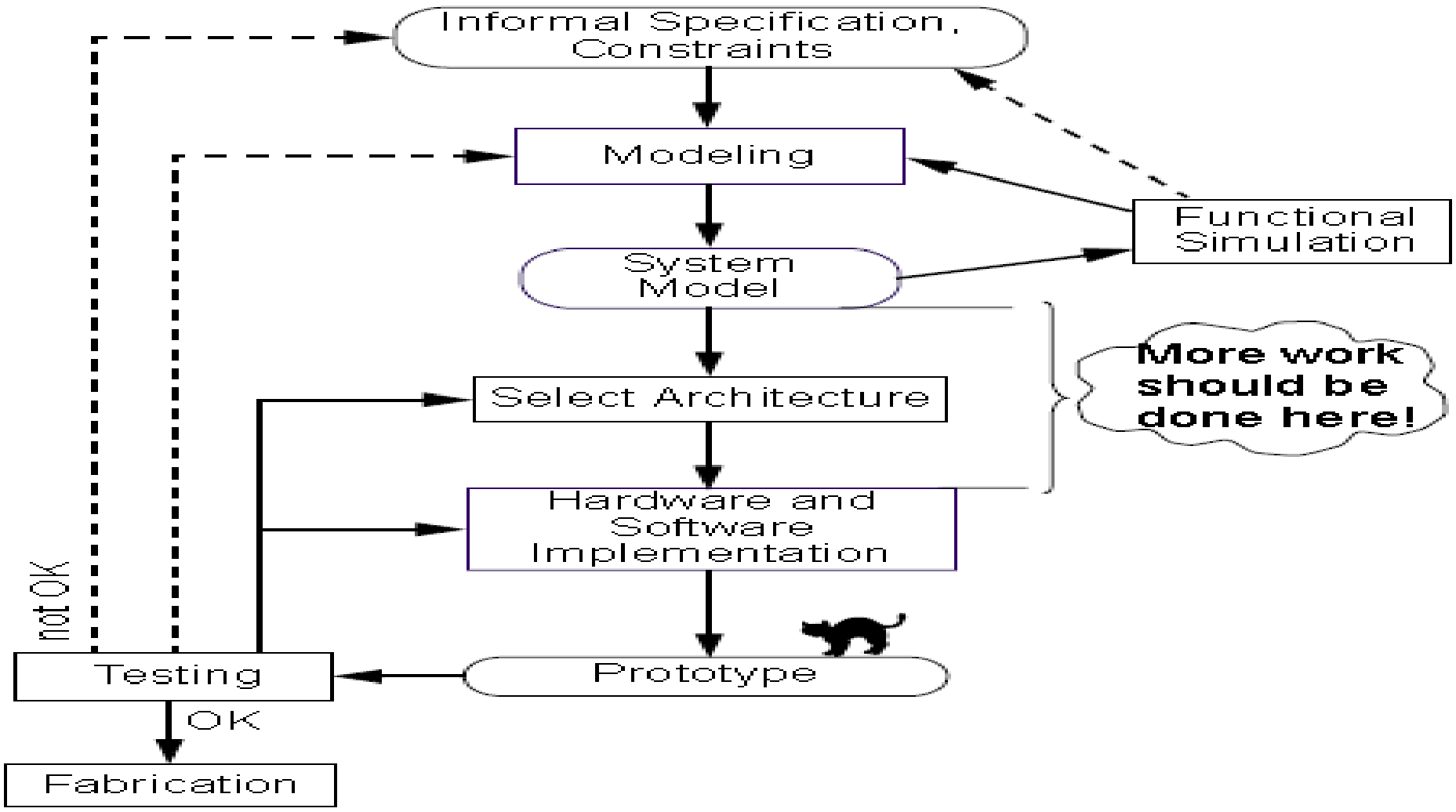
- Put together the components.
  - Many bugs appear only at this stage.
- Have a plan for integrating components to uncover bugs quickly, test as much functionality as early as possible.

Embedded system design can face many challenges, including:

- **Resource constraints**: Embedded systems have limited memory, storage, and network capabilities.
- **Real-time performance**: Embedded systems need to perform in real-time.
- **Debugging**: It can be difficult to debug embedded systems because of limited visibility into the system's internal state.
- **Reliability and robustness**: Embedded systems need to be reliable and robust.
- **Security**: Embedded systems need to be secure.
- **Scalability and flexibility**: Embedded systems need to be scalable and flexible.
- **Integration with other systems**: Embedded systems need to integrate with other systems.
- **Cost constraints**: Embedded systems need to be cost-effective.
- **Development time and tools**: Embedded systems can be challenging to develop due to time constraints and tools

https://www.rsystems.com/blogs/top-10-challenges-in-embedded-system-design-and-their-solutions/

The Traditional Design Flow (cont'd)



A phase representation of the embedded design life cycle:

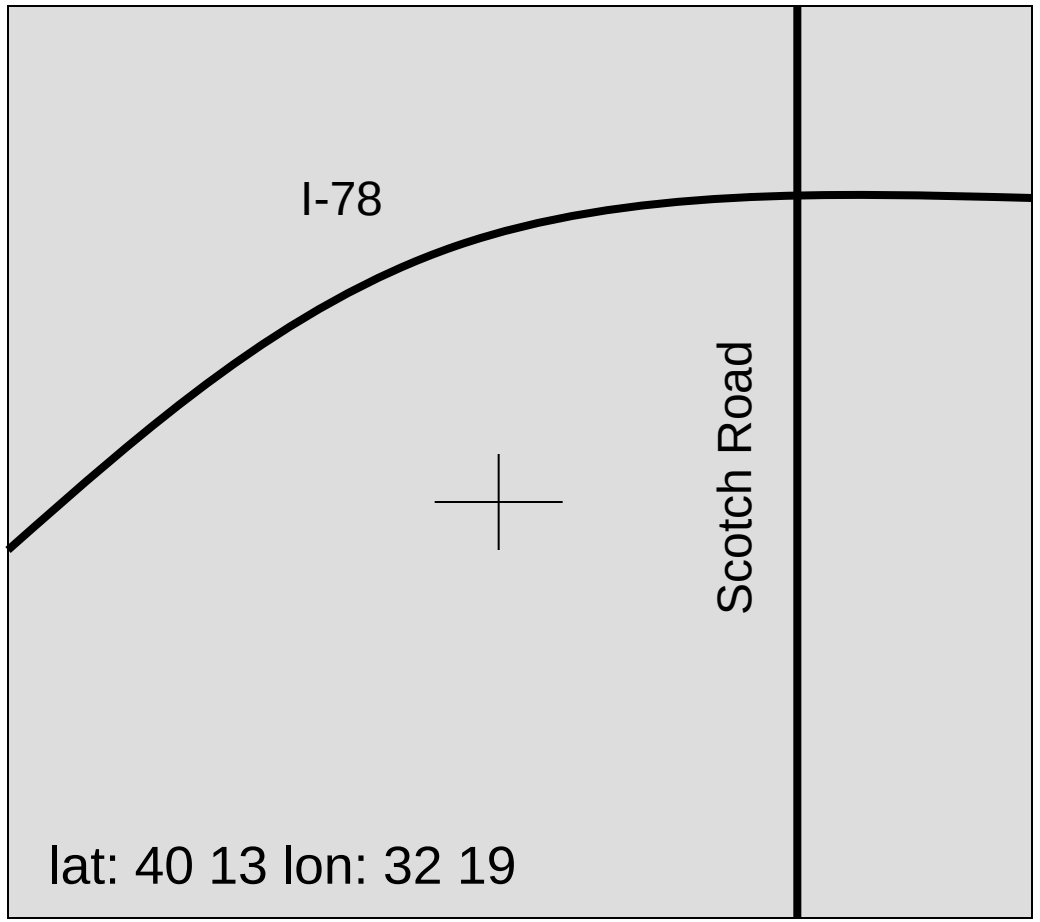
Time flows from the left and proceeds through seven phases:

- Product specification
- Partitioning of the design into its software and hardware components
- Iteration and refinement of the partitioning
- Independent hardware and software design tasks
- Integration of the hardware and software components
- Product testing and release
- On-going maintenance and upgrading

https://apptread.com/guides/embedded-system-development-life-cycle/

Example: GPS moving map requirements

- Moving map obtains position from GPS, paints map from local database.



Requirements form

name
purpose
inputs
outputs
functions
performance
manufacturing cost
power
physical size/weight

- **Type of data**
- **Data character tics**
- **Types of IO devices**

GPS moving map requirements form

Name	GPS moving map
Purpose	Consumer-grade moving map for driving use
Inputs	Power button, two control buttons
Outputs	Back-lit LCD display 400 × 600
Functions	Uses 5-receiver GPS system; three user-selectable resolutions; always displays current latitude and longitude
Performance	Updates screen within 0.25 seconds upon movement
Manufacturing cost	$30
Power	100 mW
Physical size and weight	No more than 2" × 6," 12 ounces

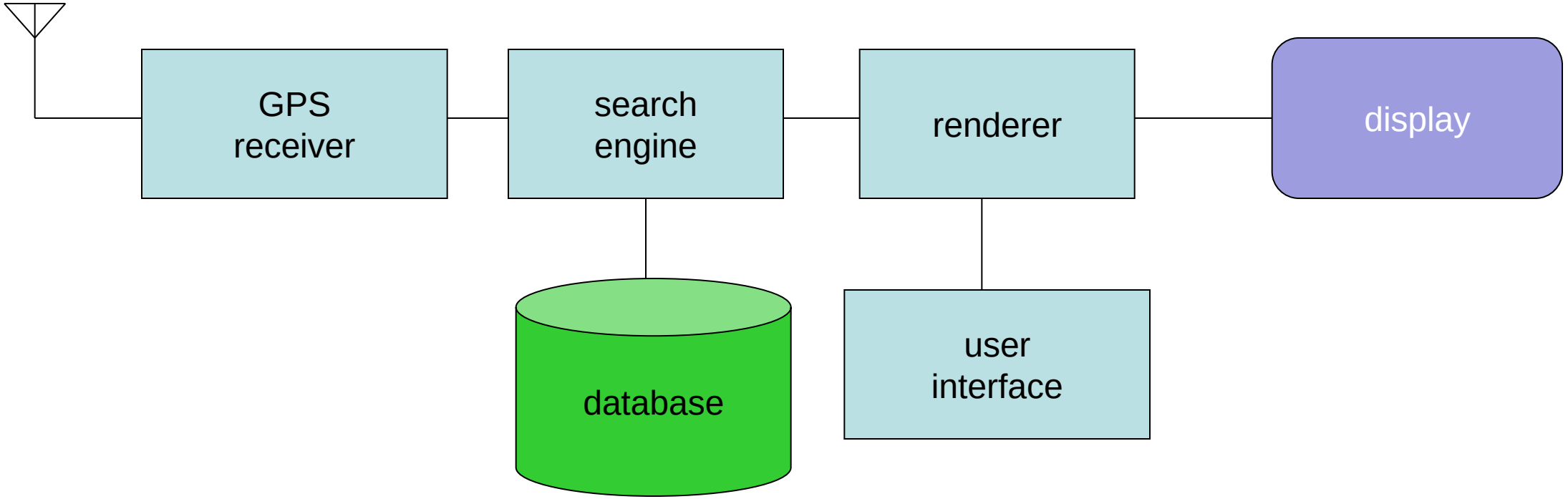
GPS specification

- A more precise description of the system:
  - should not imply a particular architecture;
  - provides input to the architecture design process.
- May include functional and non-functional elements.
- May be executable or may be in mathematical form for proofs.
- Should include:
  - **What is received from GPS;**
  - **map data;**
  - **user interface;**
  - **operations required to satisfy user requests;**
  - **background operations needed to keep the system running.**

Architecture design

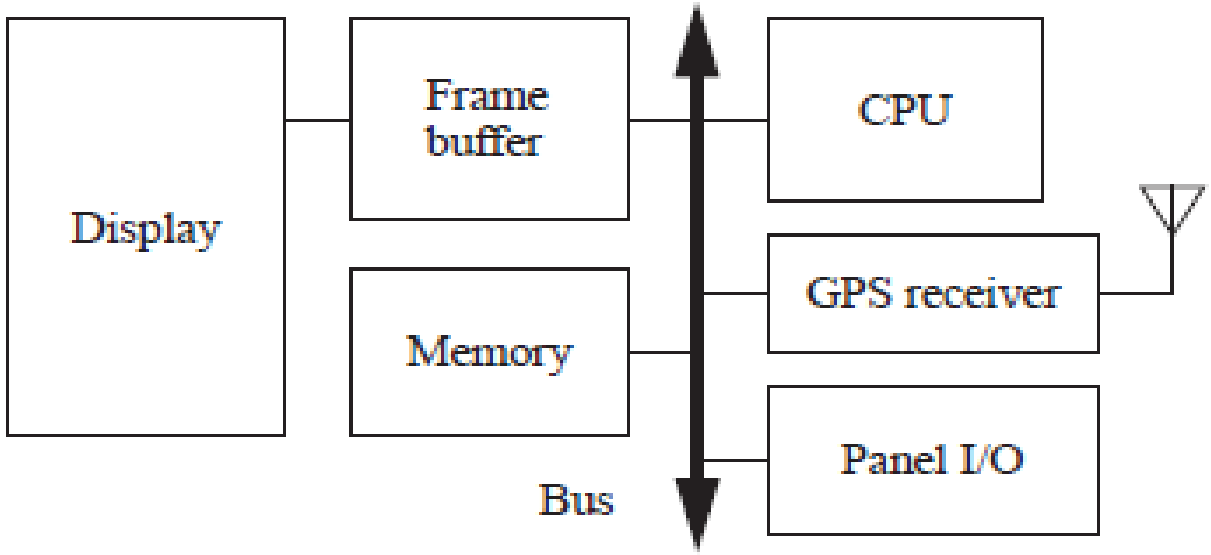
- What major components go satisfying the specification?
- **Hardware components:**
  - CPUs, peripherals, etc.
- **Software components:**
  - major programs and their operations.
- Must take into account functional and non-functional specifications.

GPS moving map block diagram



- **The receiver uses four satellites to compute latitude, longitude, altitude, and time–**
- **Three satellites**: Determine a position on Earth
- **One satellite**: Adjusts for the error in the receiver's clock.

Hardware and software architectures for the moving map.

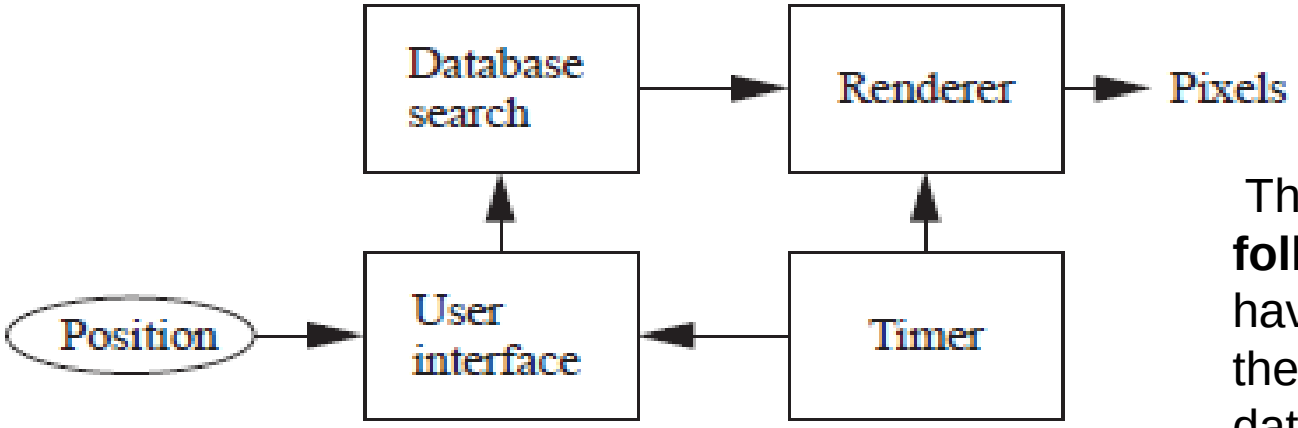


Hardware

The hardware block diagram clearly shows that we have one central CPU surrounded by memory and I/O devices.

In particular, we have chosen to **use two memories:**

- a **frame buffer** for the pixels to be displayed
- a separate **program/data memory** for general use by the CPU.



Software

The software block diagram **fairly closely follows the system block diagram,** but we have added a timer to control when we read the buttons on the user interface and render data onto the screen.

. Automotive Airbag System

An airbag system is a real-time safety-critical embedded system used to deploy airbags during collisions.

EDLC Explanation:

1. **Requirement Analysis**:
   Define collision detection, deployment timing, and safety standards.
2. **Specification**:
   Specify accelerometer sensors, microcontrollers, and safety mechanisms for airbag deployment.
3. **Architecture Design**:
   Design hardware for sensors, controllers, and deployment systems with minimal delays.
4. **Implementation**:
   Program firmware for real-time collision detection and response.
5. **Testing**:
   Test for collision accuracy, response times, and hardware durability.
6. **Deployment**:
   Integrate into vehicles during manufacturing.
7. **Maintenance**:
   Regular checks during vehicle servicing and software updates.

5. Smart Home Security System

A home security system includes motion detectors, cameras, alarms, and smart locks controlled by an embedded system.

EDLC Explanation:

1.Requirement Analysis:
Define motion detection, camera feed, alerts, and remote monitoring features.

2.Specification:
List hardware (microcontroller, sensors, cameras) and connectivity (Wi-Fi, cloud).

3.Architecture Design:
Plan hardware design and software logic for motion/event detection and alarm triggering.

4.Implementation:
Develop software to process sensor data, activate alarms, and communicate with the user.

5.Testing:
Test motion detection accuracy, camera feed, and real-time notifications.

6.Deployment:
Install in homes/offices and connect with mobile apps.

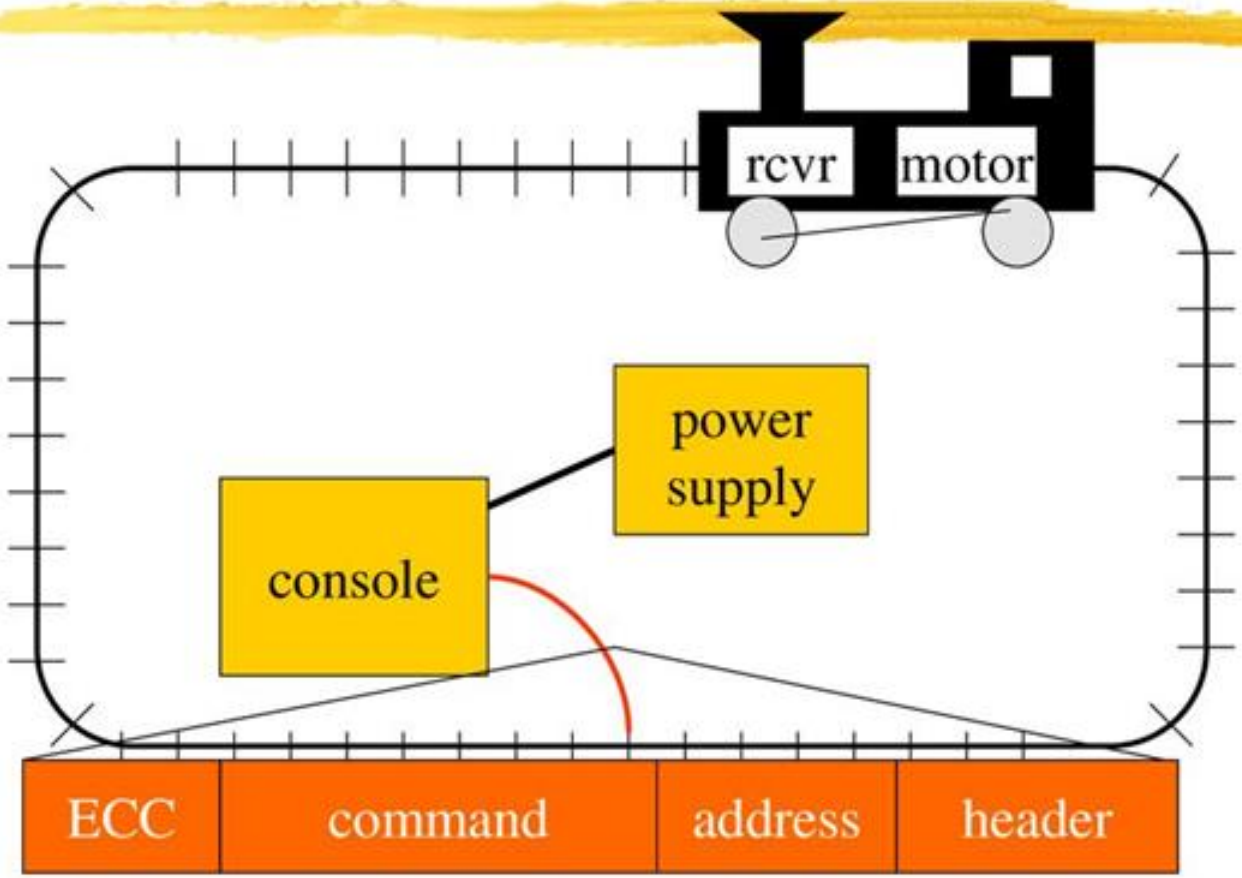
7.Maintenance:
Update software for bug fixes, security patches, and feature enhancements.

⌘Example: model train controller

Purposes of example

- ⌘Follow a design through several levels of abstraction.
- ⌘Gain experience with UML.

Model train setup



Requirements

- **The console shall be able to control up to eight trains on a single track**.
- The speed of each train shall be controllable by a **throttle to at least 63 different levels** in each direction (forward and reverse).
- There shall be an inertia control that shall allow the user to adjust the responsiveness of the train to commanded changes in speed. Higher inertia means that the train responds more slowly to a change in the throttle, simulating the inertia of a large train. **The inertia control will provide at least eight different levels.**
- There shall be an **emergency stop button**.
- **An error detection scheme will be used to transmit messages.**

Requirements form

name	model train controller
purpose	control speed of <= 8 model trains
inputs	throttle, inertia, emergency stop, train #
outputs	train control signals
functions	set engine speed w. inertia; emergency stop
performance	can update train speed at least 10 times/sec
manufacturing cost	$50
power	wall powered
physical	console comfortable for 2 hands; < 2
size/weight	lbs.

Digital Command Control

• DCC created by model railroad hobbyists, picked up by industry.

• The National Model Railroad Association to support interoperable digitally-controlled model trains.

• Defines way in which model trains, controllers communicate.

• Leaves many system design aspects open, allowing competition.

• This is a simple example of a big trend: • Cell phones, digital TV rely on standards.

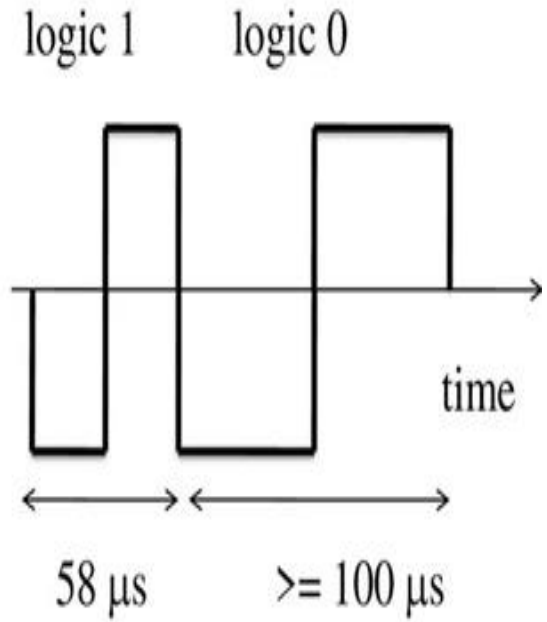
DCC documents Standard

➢ S-9.1**,** **DCC Electrical Standard**. Defines how bits are encoded on the rails.

➢S-9.2, **DCC Communication Standard**. Defines packet format and semantics.

DCC electrical standard

- Voltage moves around the power supply voltage; adds no DC component.
- 1 is 58 μs, 0 is at least 100 μs.



DCC communication standard

- Basic packet format: PSA(sD)+E.
- P: preamble = 1111111111.
- S: packet start bit = 0.
- A: address data byte.
- s: data byte start bit.
- D: data byte (data payload).
- E: packet end bit = 1.

DCC packet types

- Baseline packet: minimum packet that must be accepted by all DCC implementations.
  - Address data byte gives receiver address.
  - Instruction data byte gives basic instruction.
  - Error correction data byte gives ECC.

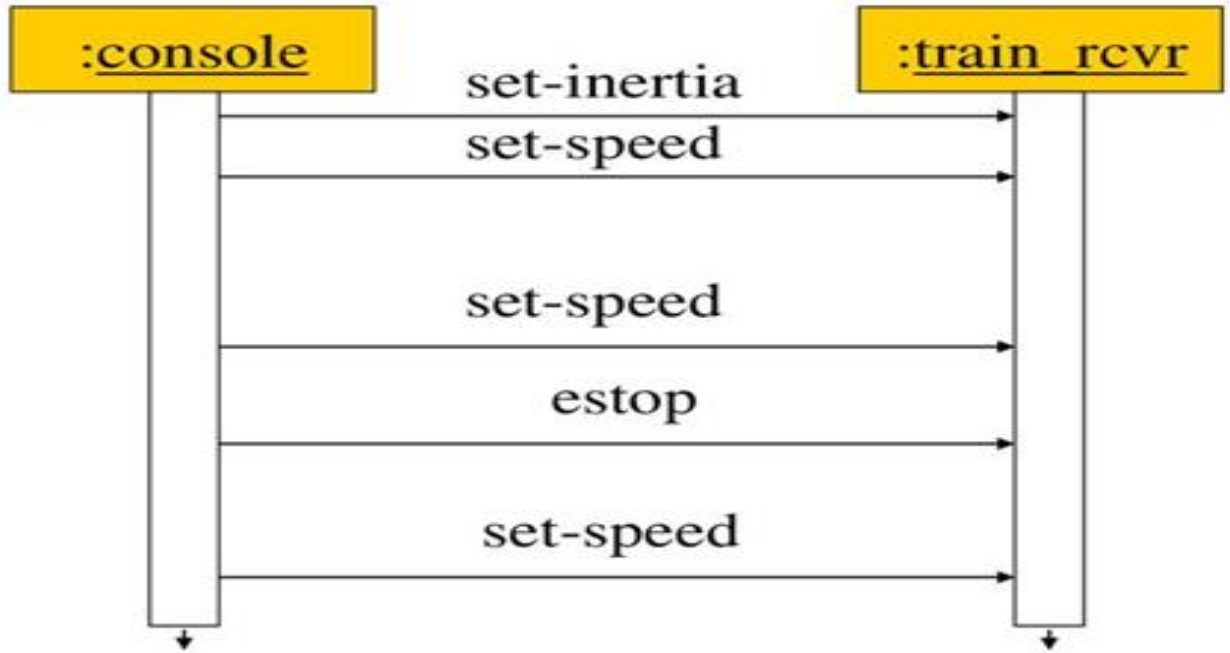
Conceptual specification

- Before we create a detailed specification, we will make an initial, simplified specification.
  - Gives us practice in specification and UML.
  - Good idea in general to identify potential problems before investing too much effort in detail.

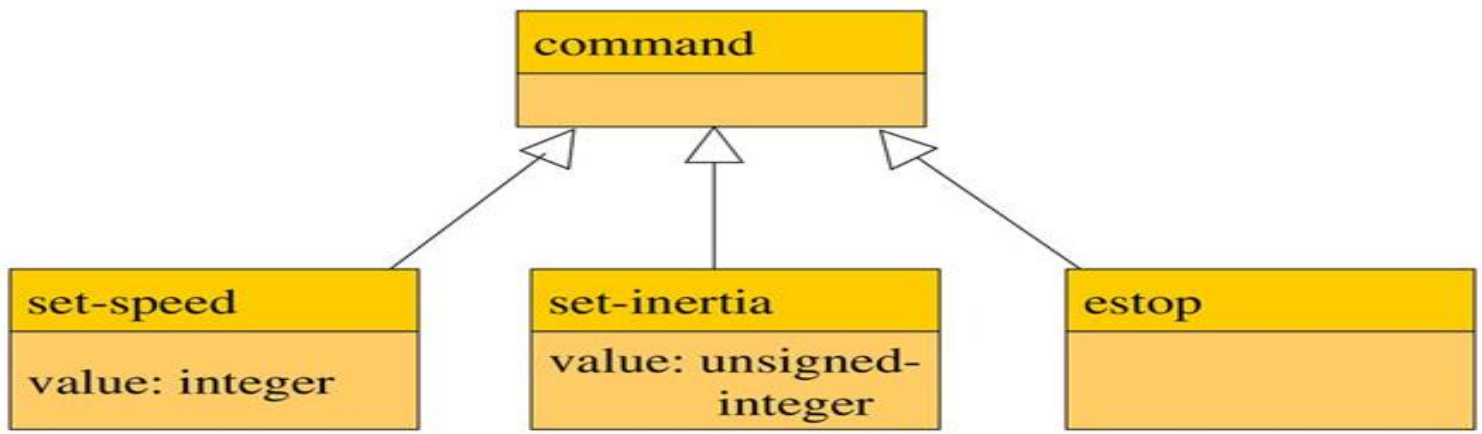
Basic system commands

command name	parameters
set-speed	speed (positive/negative)
set-inertia	inertia-value (non-negative)
estop	none

Typical control sequence



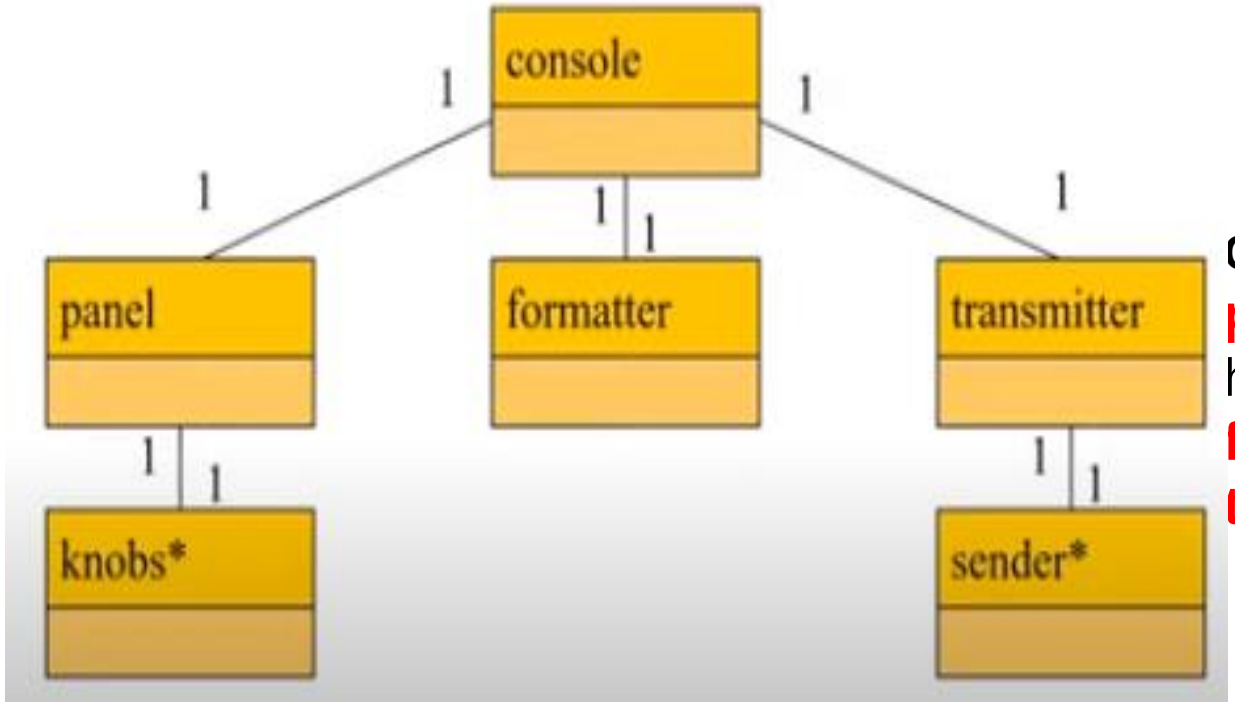
Message classes



System structure modeling

- Some classes define non-computer components.
  - Denote by *name.
- Choose important systems at this point to show basic relationships.

Console system classes



Major subsystem roles

- Console:
  - read state of front panel;
  - format messages;
  - transmit messages.
- Train:
  - receive message;
  - interpret message;
  - control the train.

Console class roles

panel: describes analog knobs and interface hardware.

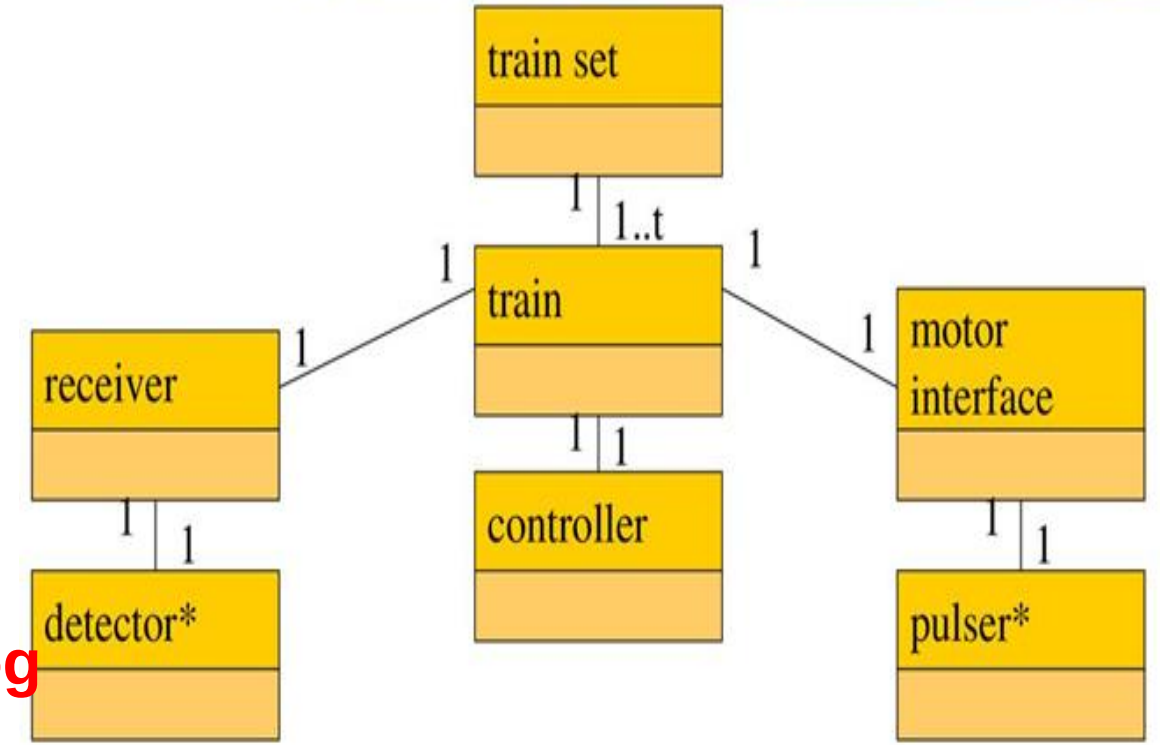
formatter: turns knob settings into bit streams.

transmitter: sends data on track.

Train class roles

- receiver: digitizes signal from track.
- controller: interprets received commands and makes control decisions.
- motor interface: generates signals required by motor.

Train system classes



Two classes to represent analog components:

- **Detector*** detects analog signals on the track and converts them into digital form.
- **Pulser*** turns digital commands into the analog signals required to control the motor speed.

The Console class describes the command unit's front panel, which contains the analog knobs and hardware to interface to the digital parts of the system.

The **Formatter** class includes behaviors that know how to read the panel knobs and creates a bit stream for the required message.

• The **Transmitter** class interfaces to analog electronics to send the message along the track.

Knobs* describes the actual analog knobs, buttons, and levers on the control panel.

Sender* describes the analog electronics that send bits along the track.

Likewise, the Train makes use of three other classes that define its components:

The Receiver class knows how to turn the analog signals on the track into digital form.

The Controller class includes behaviors that interpret the commands and figures out how to control the motor.

The Motor interface class defines how to generate the analog signals required to control the motor.

Two classes to represent analog components:

■**Detector*** detects analog signals on the track and converts them into digital form.

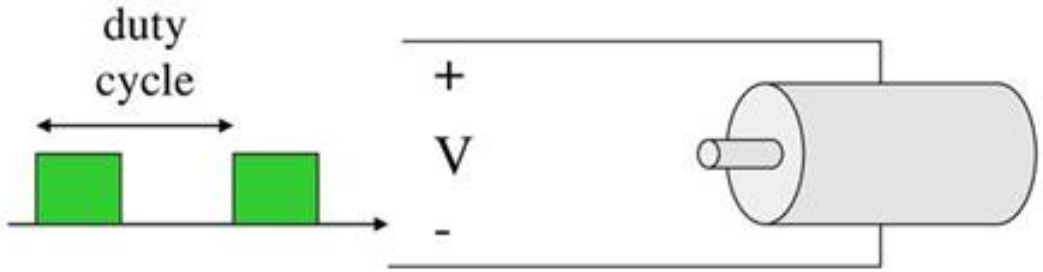
■**Pulser*** turns digital commands into the analog signals required to control the motor speed.

Detailed specification

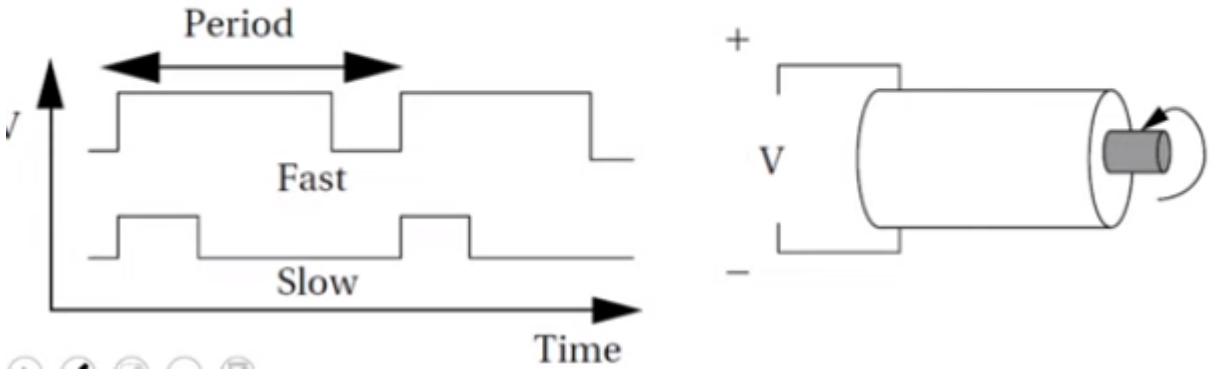
- We can now fill in the details of the conceptual specification:
  - more classes;
  - behaviors.
- Sketching out the spec first helps us understand the basic relationships in the system.

Train speed control

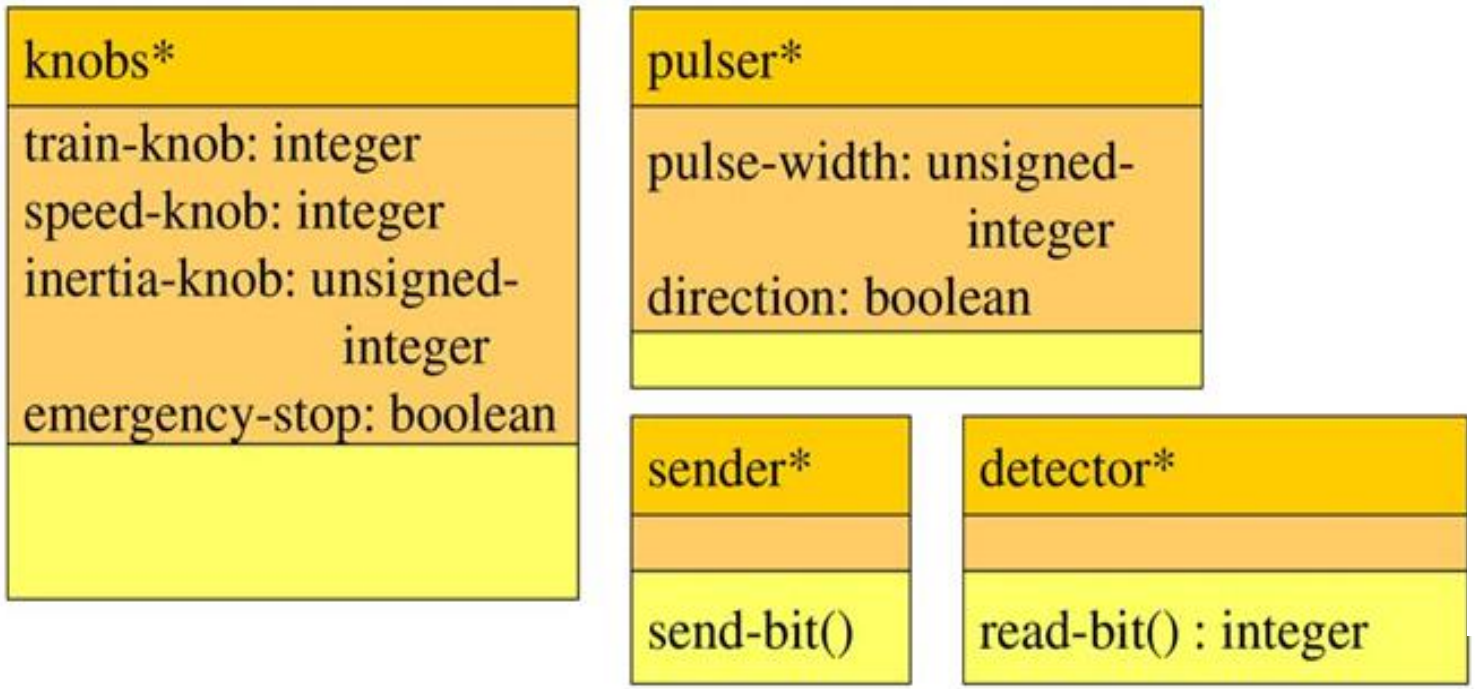
- Motor controlled by pulse width modulation:



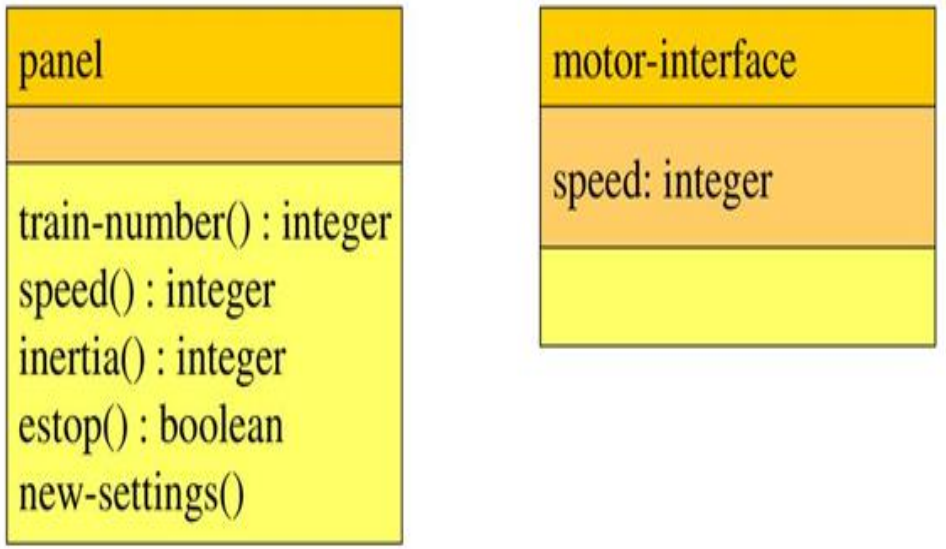
- Motor controlled by pulse width modulation:



Console physical object classes



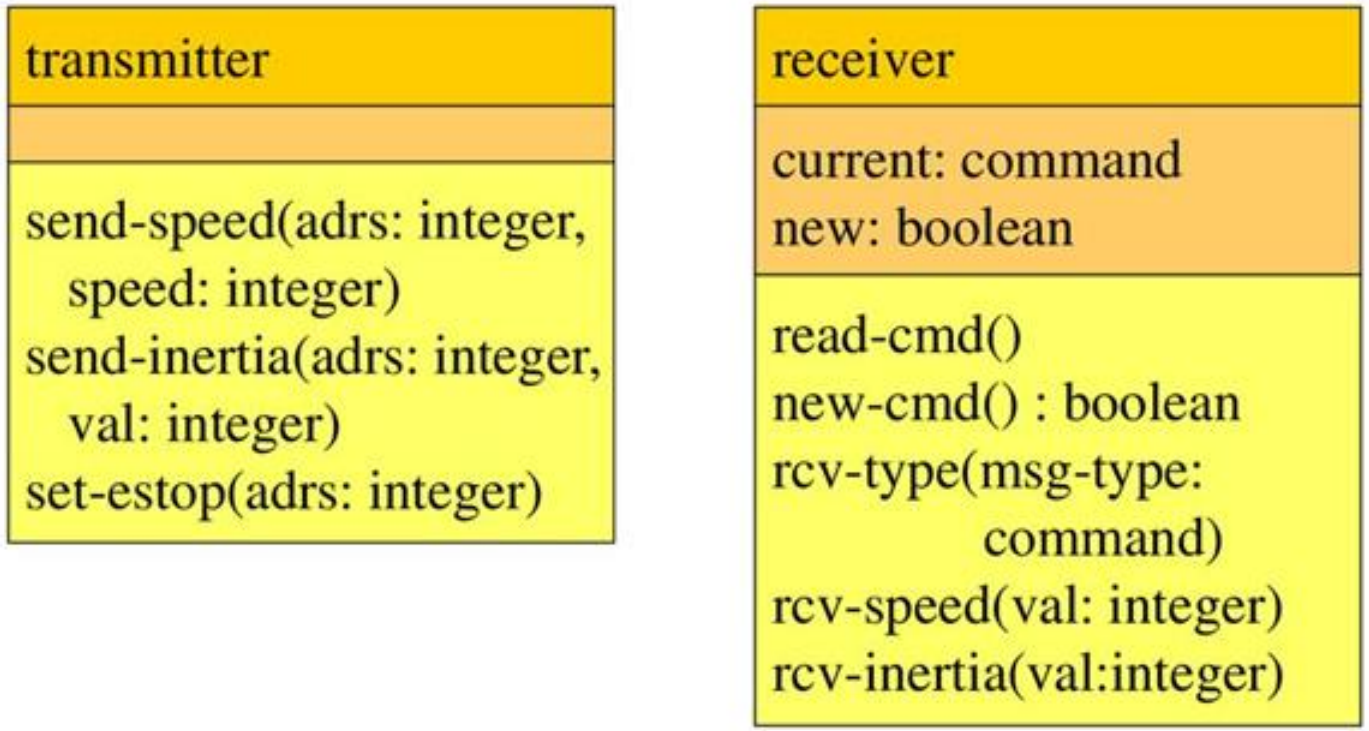
Panel and motor interface classes



Class descriptions

- panel class defines the controls.
  - new-settings() behavior reads the controls.
- motor-interface class defines the motor speed held as state.

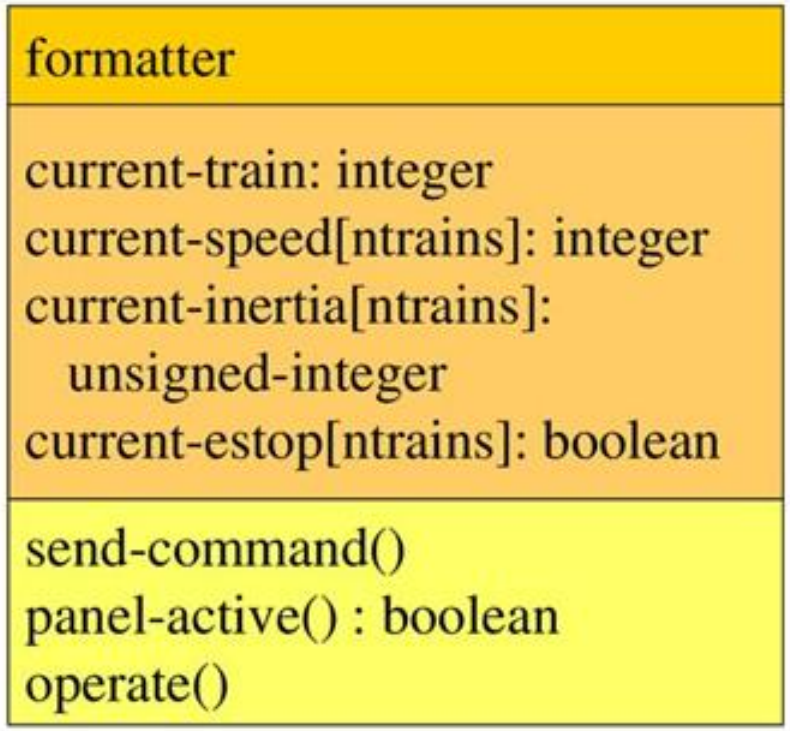
Transmitter and receiver classes



Class descriptions

- transmitter class has one behavior for each type of message sent.
- receiver function provides methods to:
  - detect a new message;
  - determine its type;
  - read its parameters (estop has no parameters).

Formatter class



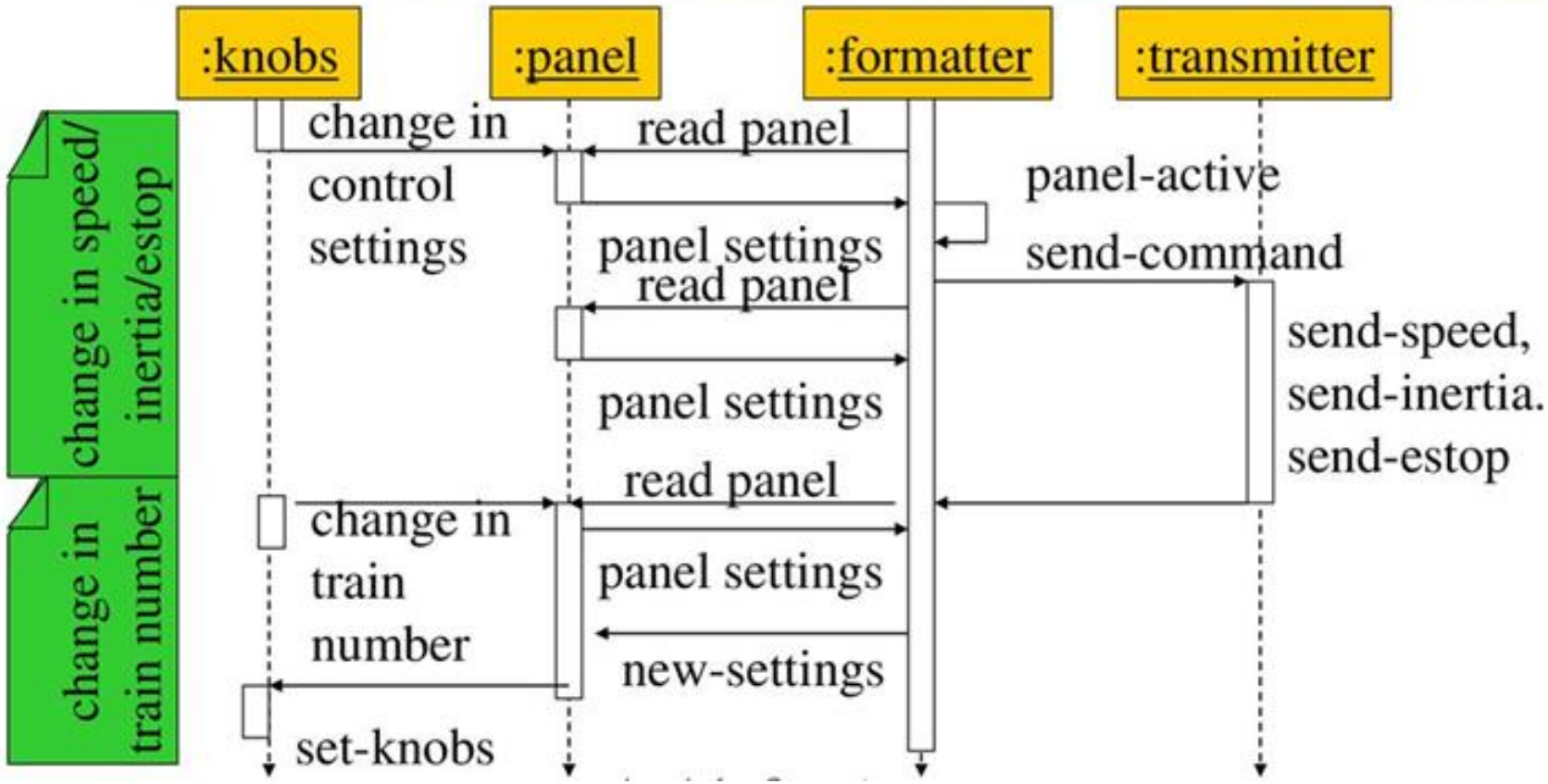
Formatter class description

- Formatter class holds state for each train, setting for current train.
- The operate() operation performs the basic formatting task.

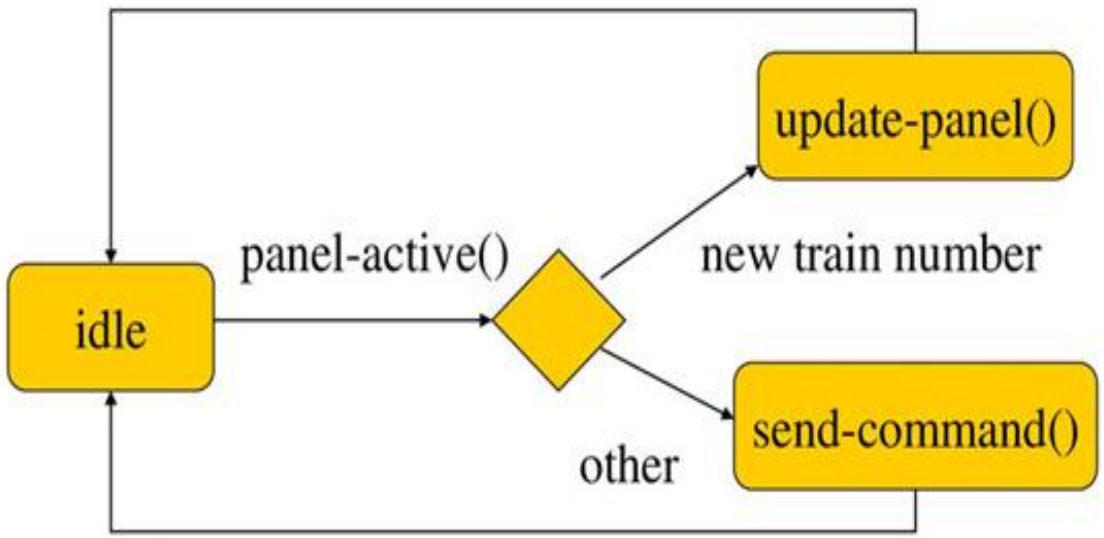
Control input cases

- Use a soft panel to show current panel settings for each train.
- Changing train number:
  - must change soft panel settings to reflect current train's speed, etc.
- Controlling throttle/inertia/estop:
  - read panel, check for changes, perform command.

Control input sequence diagram



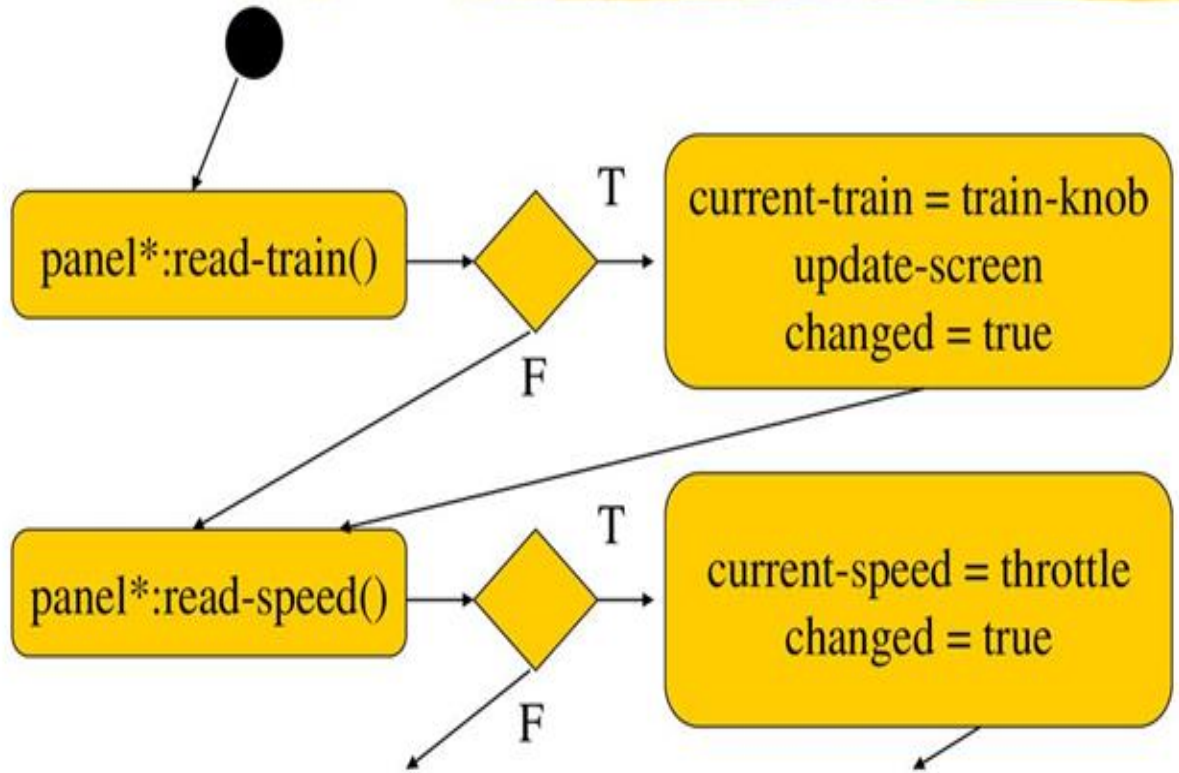
Formatter operate behavior



Controller class

controller
current-train: integer current-speed[ntrains]: integer current-direction[ntrains]: boolean current-inertia[ntrains]: unsigned-integer
operate() issue-command()

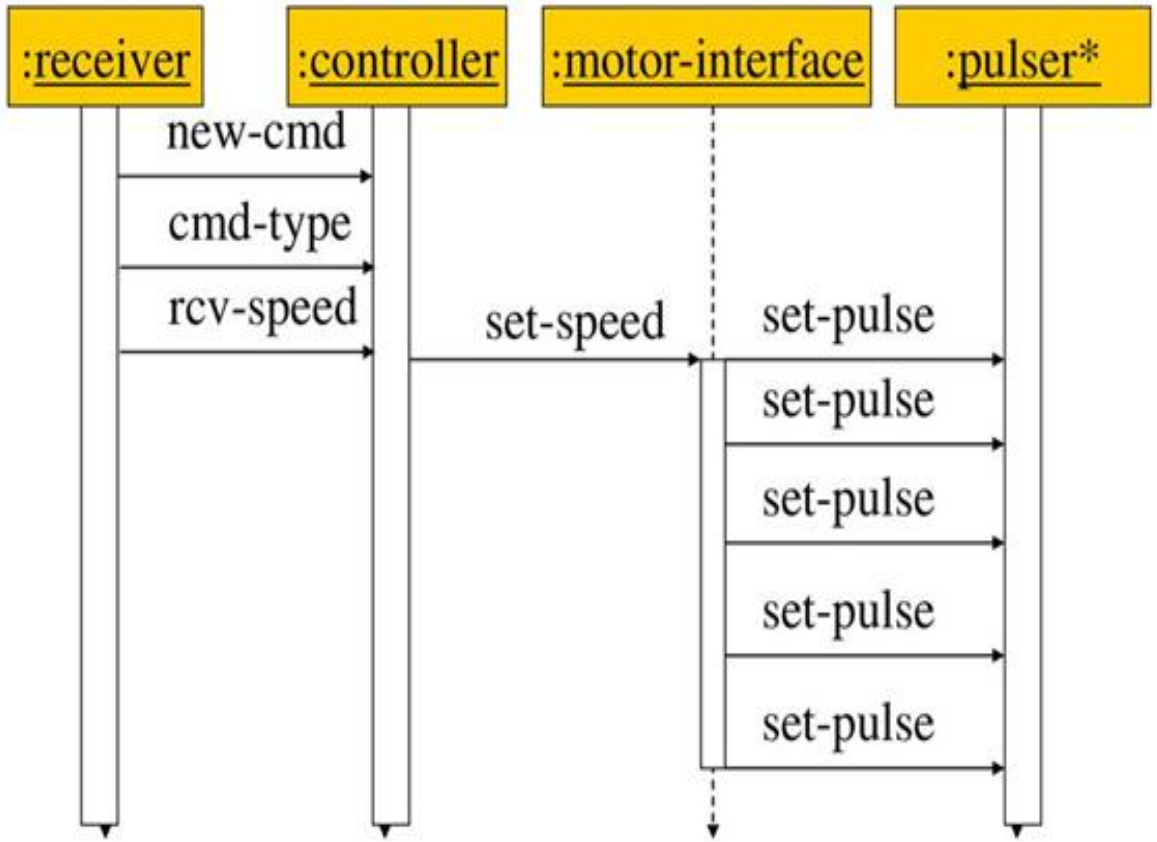
Panel-active behavior



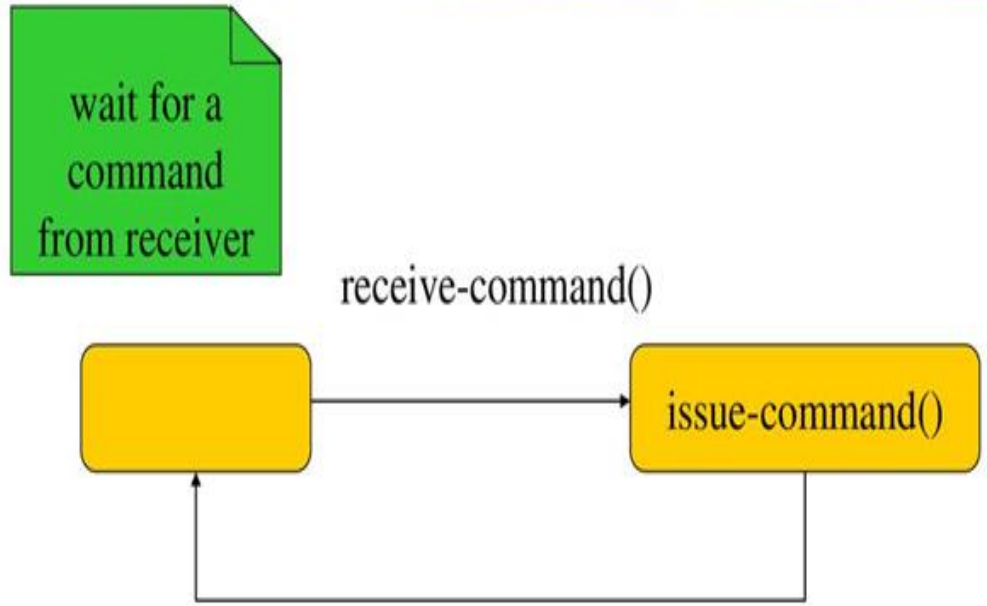
Setting the speed

- Don't want to change speed instantaneously.
- Controller should change speed gradually by sending several commands.

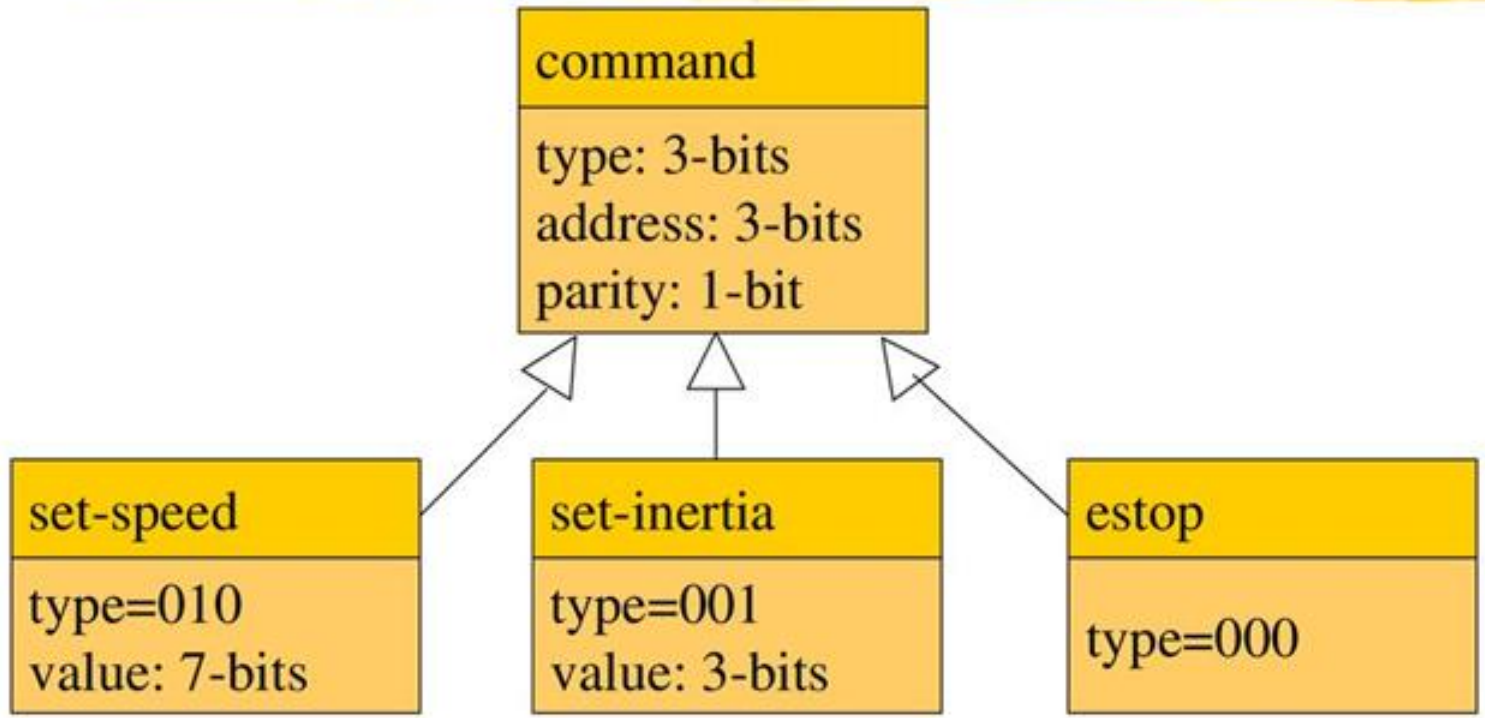
Sequence diagram for set-speed command

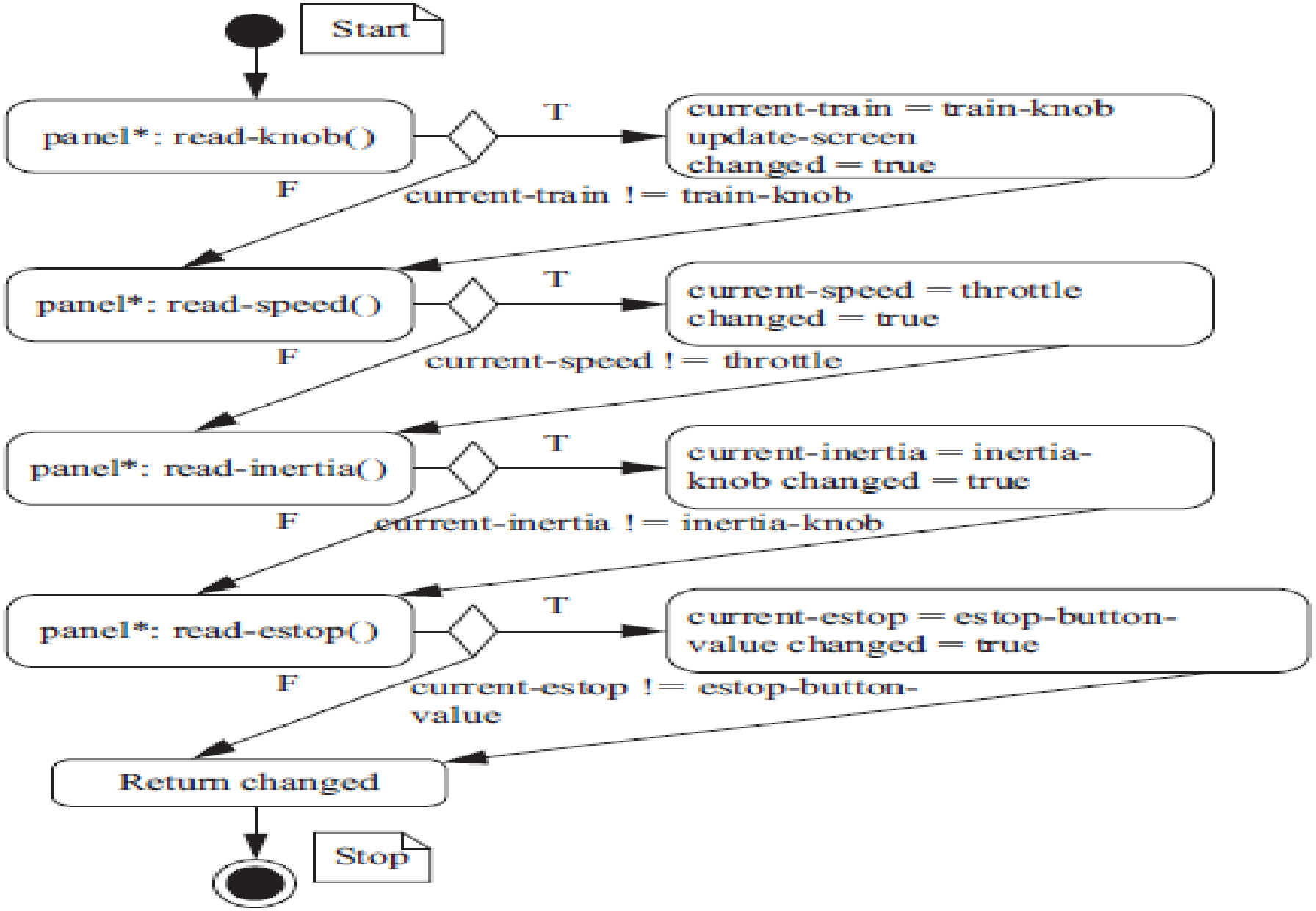


Controller operate behavior



Refined command classes





State diagram for the panel-active behavior.

Application examples

- Simple control: front panel of microwave oven, etc.
- Canon EOS 3 has three microprocessors.
  - 32-bit RISC CPU runs autofocus and eye control systems.
- Digital TV: programmable CPUs + hardwired logic.

Examples in your Daily Life-Automotive embedded systems

- ...get into your car ...

- A latest model car can have as many as 65+ processors for Engine control, Transmission Control, A/C control, Cruise control, ABS, Audio, etc
- More than 30% of the cost of a car is now in electronics
- 90% of all innovations will be based on electronic systems

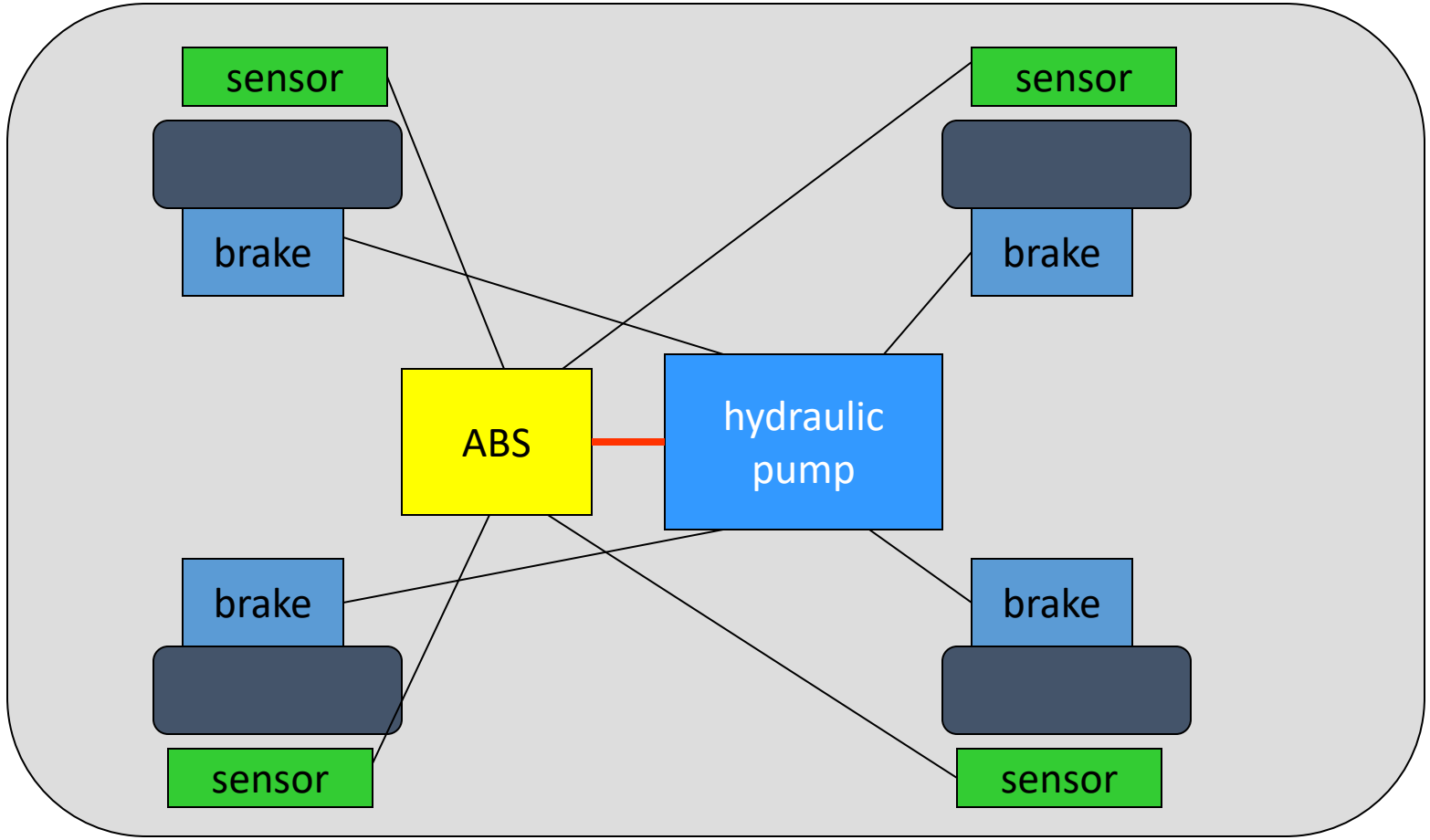
➢ Today's high-end automobile may have 100 Processors:
  4-bit microcontroller checks seat belt;
  microcontrollers run dashboard devices;
  16/32-bit microprocessor controls engine.

➢ Low-end cars use 20+ microprocessors.

BMW 850i brake and stability control system

- Anti-lock brake system (ABS): pumps brakes to reduce skidding.
- Automatic stability control (ASC+T): controls engine to improve stability.
- ABS and ASC+T communicate.
  - ABS was introduced first---needed to interface to existing ABS module.

BMW 850i, cont'd.

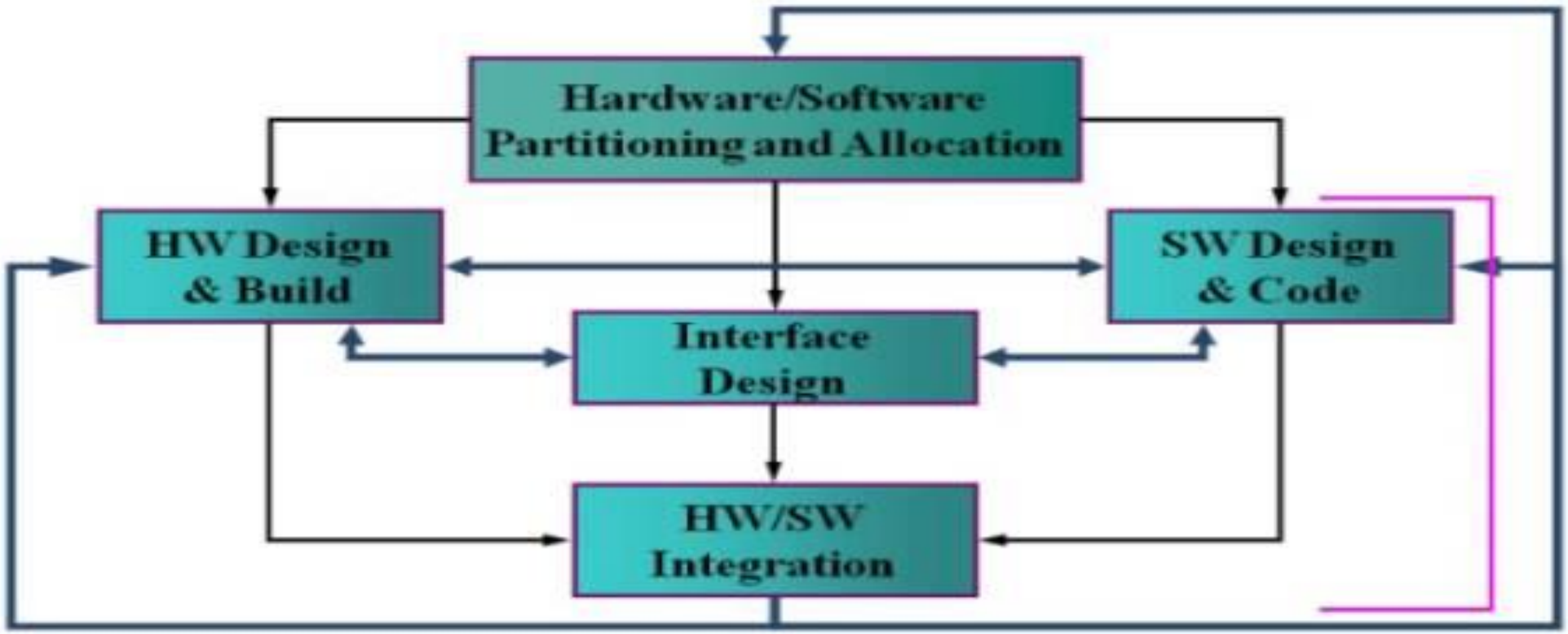


The purpose of an ABS is to temporarily release the brake on a wheel when it rotates too slowly- when a wheel stops turning, the car starts skidding and becomes hard to control. It sits between the hydraulic pump, which provides power to the brakes, and the brakes themselves as seen in the accompanying diagram. This hookup allows the ABS system to modulate the brakes to keep the wheels from locking. The ABS system uses sensors on each wheel to measure the speed of the wheel. The wheel speeds are used by the ABS system to determine how to vary the hydraulic fluid pressure to prevent the wheels from skidding.

The ASC+ T system's job is to control the engine power and the brake to improve the car's stability during maneuvers. The ASC+ T controls four different systems: **throttle, ignition timing, differential brake, and (on automatic transmission cars) gear shifting**. The ASC+T can be turned off by the driver, which can be important when operating with tire snow chains.
The ABS and ASC+ T must clearly communicate because the ASC+T interacts with the brake system. Since the ABS was introduced several years earlier than the ASC+T, it was important to be able to interface ASC+T to the existing ABS module, as well as to other existing electronic modules. The engine and control management units include the electronically controlled throttle, digital engine management, and electronic transmission control. **The ASC+T control unit has two microprocessors on two printed circuit boards, one of which concentrates on logic-relevant components and the other on performance-specific components.**

Embedded System Design

HW/SW Co-Design Methodology



Characteristics of Embedded Systems

Low power consumption	Optimized system failure rate	Little to no maintenance overhead and human involvement
Increased resistance to external factors	Minimized size, weight, and costs	Sole task completion
Continuous and uninterrupted operation	Simplistic programming	Exceptional levels of fault tolerance

TATEEDA

Characteristics of embedded systems

- Sophisticated functionality.
- Real-time operation.
- Low manufacturing cost.
- Low power
- Designed to tight deadlines by small teams.

Functional complexity

Often have to run sophisticated algorithms or multiple algorithms.
Cell phone, laser printer.
Often provide sophisticated user interfaces.

Characteristics of Embedded Systems

- Embedded systems possess certain specific characteristics.
- These characteristics are unique to each embedded system.
- Some of the important characteristics of an embedded system are:

1. Application and domain specific
2. Reactive and Real Time
3. Operates in harsh environments
4. Distributed
5. Small size and weight
6. Power concerns

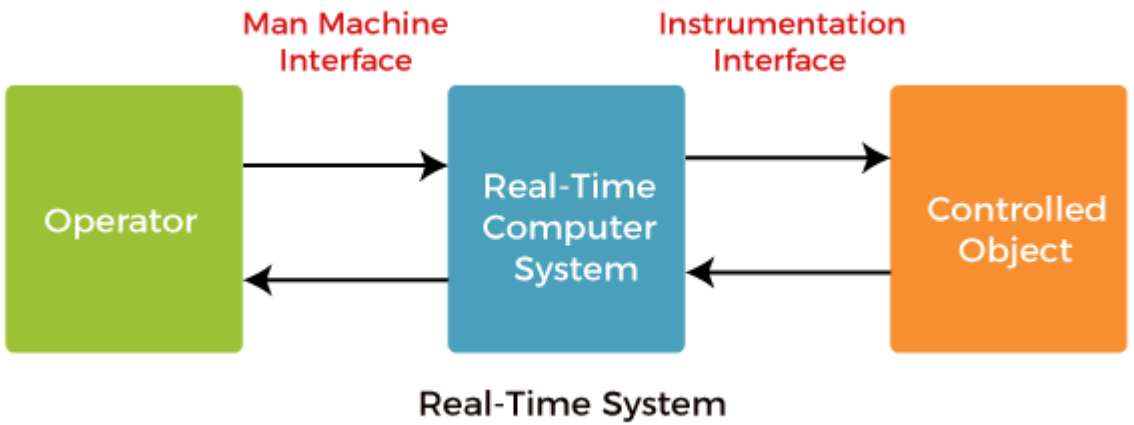
Operational Quality Attributes

- The operational quality attributes represent the relevant quality attributes related to the embedded system when it is in the operational mode or 'online' mode.
- The important operational quality attributes are:

1. Response
2. Throughput
3. Reliability
4. Maintainability
5. Security
6. Safety

Real-Time Operating Systems

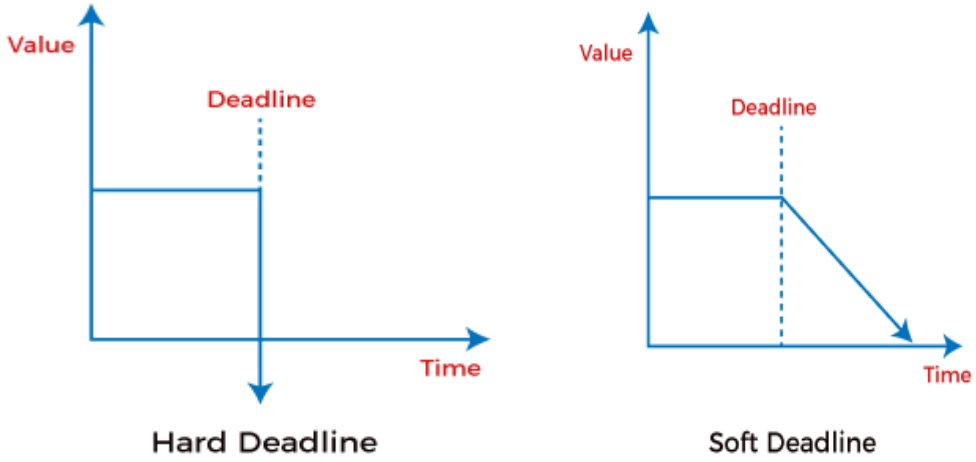
A real-time system is a time-bound system with well-defined and fixed time constraints, and processing must be done within the defined constraints; otherwise, the system will fail. In a real-time operating system, processing time requirements are measured in ***tenths of seconds***.



Real-Time System

Real-time operating systems involve a set of applications where the operations are performed on time to run the activities in an external system

Real-time operation



- Must finish operations by deadlines.
  - Hard real time: missing deadline causes failure.
  - Soft real time: missing deadline results in degraded performance.
- Many systems are multi-rate: must handle operations at widely varying rates.- Multirate embedded computing systems are very common, including automobile engines, printers, and cell phones. In all these systems, certain operations must be executed periodically, and each operation is executed at its own rate
- Difference between Hard and Soft Real-Time System

https://www.javatpoint.com/hard-and-soft-real-time-operating-system#:~:text=A%20hard%2Dreal%20time%20system,is%20considered%20to%20be%20degraded.

Some Common Characteristics

- Single-functioned
  - Executes a single program, repeatedly
- Tightly-constrained
  - Low cost, low power, small, fast, etc.
- Reactive and real-time
  - Continually reacts to changes in the system's environment
  - Must compute certain results in real-time without delay

Challenges in embedded system design

- **How much hardware do we need?**
  - How big is the CPU? Memory?
- **How do we meet our deadlines?**
  - Faster hardware or cleverer software?
- **How do we minimize power?**
  - Turn off unnecessary logic? Reduce memory accesses?
- **How do we design for upgradeability?**

  several different versions of a product in the same generation, with few or no changes -able to add features by changing software.
- **Is it secure?**
- Attacks on embedded systems can not only gain access to personal data but also cause the physical systems controlled by those embedded computers to perform dangerous acts.

Challenges, etc.

- Does it really work?
  - Is the specification correct?
  - Does the implementation meet the spec?
  - How do we test for real-time characteristics?
  - How do we test on real data?
- How do we work on the system?
  - Observability, controllability?
  - What is our development platform?

Design challenge – optimizing design metrics

- Obvious design goal:
  - Construct an implementation with desired functionality
- Key design challenge:
  - Simultaneously optimize numerous design metrics
- Design metric
  - A measurable feature of a system's implementation
  - Optimizing design metrics is a key challenge

Design challenge – optimizing design metrics

- **DESIGN METRICS OF EMBEDDED SYSTEMS**
  - NRE cost (Non-Recurring Engineering cost):

    The one-time monetary cost of designing the system. Once the system is designed, any no of units can be manufactured without incurring any additional design cost.
  - Unit cost:
    - the monetary cost of manufacturing each copy of the system, excluding NRE cost
  - Size:
    - The physical space required by the system.
    - Software - Bytes
    - Hardware – gates and transistors
  - Performance:
    - The execution time or throughput of the system

Design challenge – optimizing design metrics

- Common metrics (continued)
  - Power: The amount of power consumed by the system
  - Flexibility:
    - The ability to change the functionality of the system without incurring heavy NRE cost
  - Time-to-prototype:
    - The time needed to build a working version of the system
  - Time-to-market:
    - The time required to develop a system to the point that it can be released and sold to customers
  - Maintainability:
    - The ability to modify the system after its initial release.
  - Correctness, safety

What does “performance” mean?

- In general-purpose computing, performance often means average-case, may not be well-defined.
- In real-time systems, performance means meeting deadlines.
  - Missing the deadline by even a little is bad.
  - Finishing ahead of the deadline may not help.

Performance of a system is a measure of how long the system takes to execute our desired task

Main measures of performance

- Latency Time b/w start of task of task execution to end
- Throughput No of task that can be processed per unit time
- Speed Up is a common method of comparing the performance of two systems.

Why is Design of Embedded Systems Difficult?

- High Complexity
- Strong time and power constraints
- Low cost
- Short time to market
- Safety critical systems
- In order to achieve all these requirements, systems have to be highly optimized.
- *Both hardware and software aspects have to be considered simultaneously*!

Security, safety

- Security: system's ability to prevent malicious attacks.
- Integrity: maintenance of proper data values.
- Privacy: no unauthorized releases of data.

- Safety: no harmful releases of energy.
  - No crashes, accidents, etc.

Embedded processor technology

Book : Frank Vahid

Processor Technology (cont.)

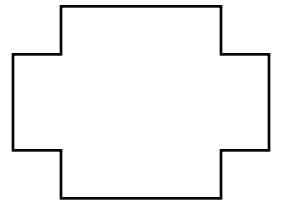
- Processors vary in their customization for the problem at hand

Desired functionality

```
total = 0
for i = 1 to N  loop
   total += M[i]
end loop
```

General-purpose processor

Application-specific processor



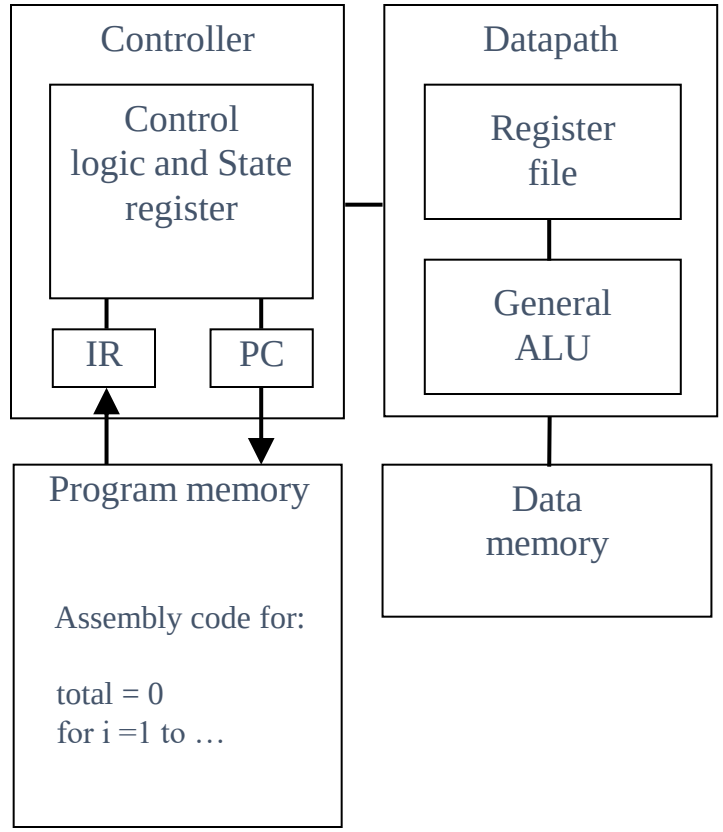
Single-purpose processor

Embedded processor technology

- General-purpose processors -- software
- Single-purpose processors – hardware
- Application-specific processors

General-Purpose Processors

- Programmable device used in a variety of applications
  - Also known as “microprocessor”
- Features
  - Program memory
  - General datapath with large register file and general ALU
- User benefits
  - Low time-to-market and NRE costs
  - High flexibility
- “Intel/AMD” the most well-known, but there are hundreds of others



Embedded processor technology
General-purpose processors -- software

- The designer of a general-purpose processor builds a device suitable for a variety of applications, to maximize the number of devices sold
- One feature of such a processor is a program memory
- Another feature is a general datapath – the datapath must be general enough to handle a variety of computations, so typically has a large register file and one or more general-purpose arithmetic-logic units (ALUs)

Embedded processor technology
General-purpose processors -- software

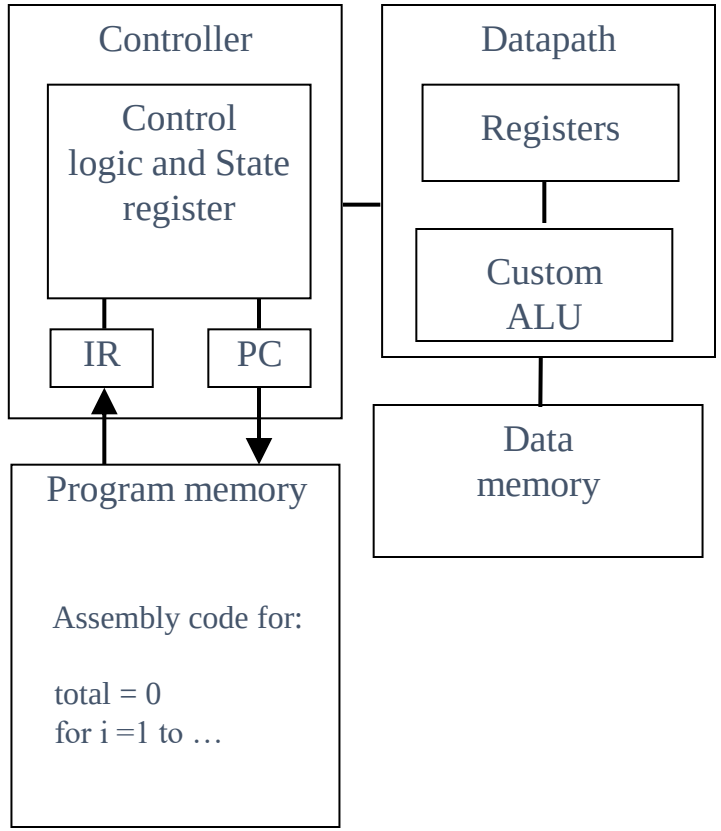
- **Design time and NRE cost are low,** because the designer must only write a program, but need not do any digital design.
- **Flexibility is high**, because changing functionality requires only changing the program.
- **Unit cost may be relatively low in small quantities**, since the processor manufacturer sells large quantities to other customers and hence distributes the NRE cost over many units.
- **Performance may be fast for computation-intensive applications**, if using a fast processor, due to advanced architecture features and leading edge IC technology.

Design-metric drawbacks.

- Unit cost may be too high for large quantities.
- Performance may be slow for certain applications. Size and power may be large due to unnecessary processor hardware.

Application-Specific Processors

- Programmable processor optimized for a particular class of applications having common characteristics
  - Compromise between general-purpose and single-purpose processors
- Features
  - Program memory
  - Optimized datapath
  - Special functional units
- Benefits
  - Some flexibility, good performance, size and power

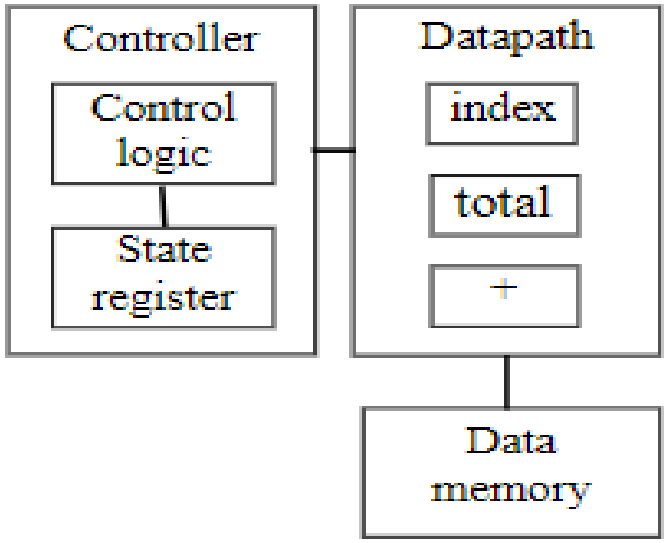


Application-specific processors

- An application-specific instruction-set processor (or ASIP) can serve as a compromise between the above processor options.
- An ASIP is designed for a particular class of applications with common characteristics, such as digital-signal processing, telecommunications, embedded control, etc.
- The designer of such a processor can optimize the datapath for the application class, perhaps adding special functional units for common operations, and eliminating other infrequently used units
- Embedded system can provide the benefit of flexibility while still achieving good performance, power and size.
- However, such processors can require large NRE cost to build the processor itself, and to build a compiler, if these items don’t already exist

Single-purpose processors

- Digital circuit designed to execute exactly one program
  - a.k.a. coprocessor, accelerator or peripheral
- Features
  - Contains only the components needed to execute a single program
  - No program memory
- Benefits
  - Fast
  - Low power
  - Small size



Embedded processor technology
Single-purpose processors -- hardware

- Single-purpose processors -- hardware A single-purpose processor is a digital circuit designed to execute exactly one program. For example, consider the digital camera
- All of the components other than the microcontroller are single-purpose processors. The JPEG codec, for example, executes a single program that compresses and decompresses video frames. An embedded system designer creates a single-purpose processor

Embedded processor technology
Single-purpose processors -- hardware

- Performance may be fast,
- size and power may be small, and
- unit-cost may be low for large quantities, while design time and NRE costs may be high,
- flexibility is low,
- unit cost may be high for small quantities, and performance may not match general-purpose processors for some applications.

Independence of Processor Technologies

- Basic tradeoff
  - General vs. custom
  - With respect to processor technology or IC technology
  - The two technologies are independent

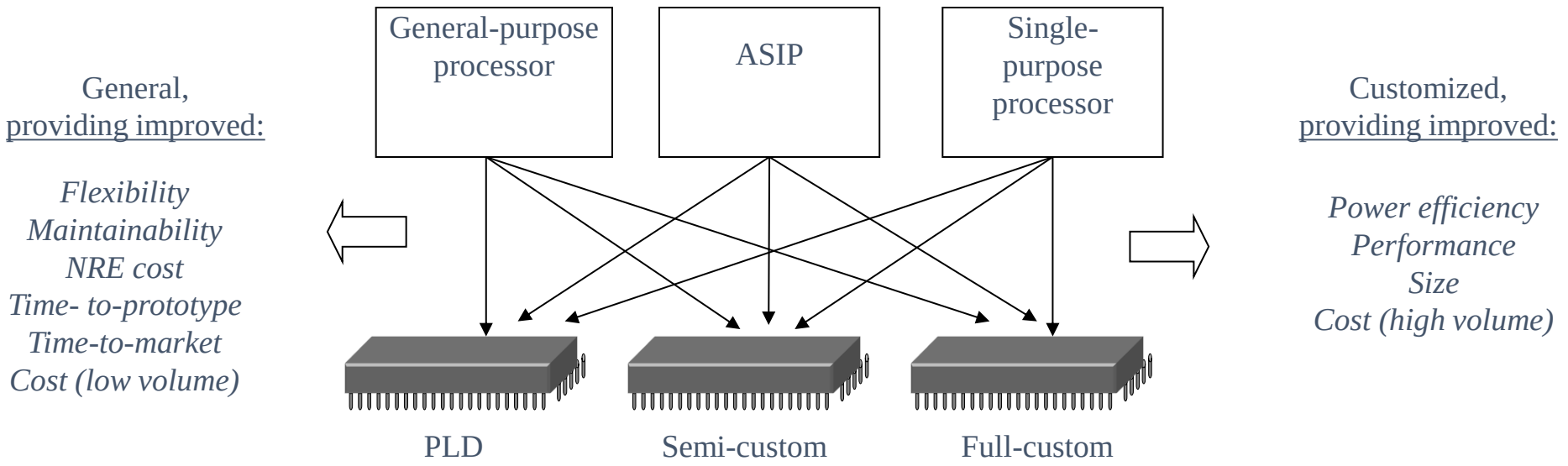
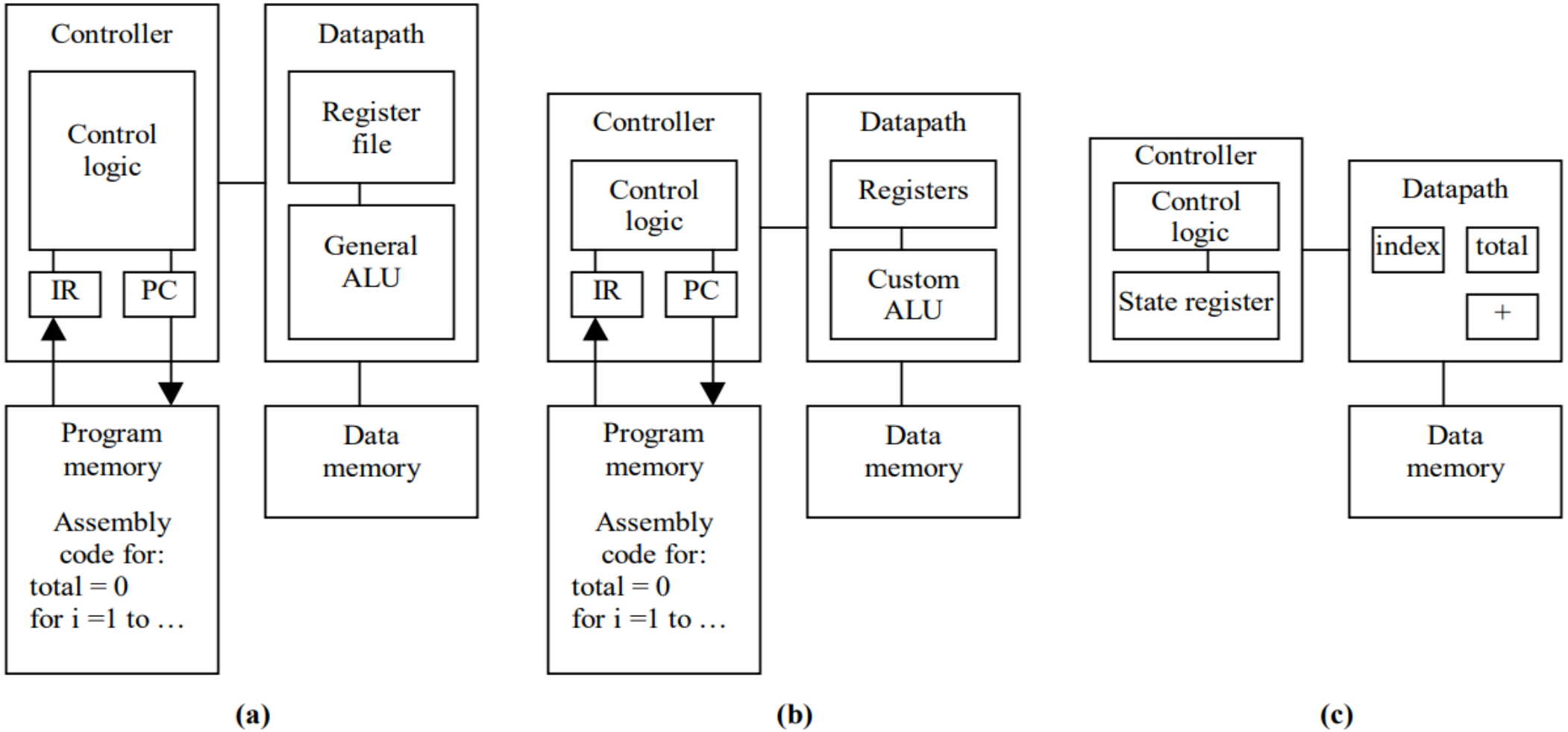


Figure 1.6: Implementing desired functionality on different processor types: (a) general-purpose, (b) application-specific, (c) single-purpose.



Why use microprocessors?

- Alternatives: field-programmable gate arrays (FPGAs), custom logic, etc.
- Microprocessors are often very efficient: can use same logic to perform many different functions.
- Microprocessors simplify the design of families of products.

The performance paradox

- Microprocessors use much more logic to implement a function than does custom logic.
- But microprocessors are often at least as fast:
  - heavily pipelined;
  - large design teams;
  - aggressive VLSI technology.

Power

- Custom logic uses less power, but CPUs have advantages:
  - Modern microprocessors offer features to help control power consumption.
  - Software design techniques can help reduce power consumption.
- Heterogeneous systems: some custom logic for well-defined functions, CPUs+software for everything else.

Platforms

- Embedded computing platform: hardware architecture + associated software.
- Many platforms are multiprocessors.
- Examples:
  - Single-chip multiprocessors for cell phone baseband.
  - Automotive network + processors.

The physics of software

- Computing is a physical act.
  - Software doesn’t do anything without hardware.
- Executing software consumes energy, requires time.
- To understand the dynamics of software (time, energy), we need to characterize the platform on which the software runs.

Internet-of-Things (IoT) system

- Combines sensing, actuating, computing, communication.
  - Some links in the network are often wireless.
- Example: manufacturing plant.

Cyber-physical systems

A cyber-physical system is one that combines physical devices, known as the plant, with computers that control the plant. The embedded computer is the cyber part of the cyber-physical system.

- A physical system that tightly interacts with a computer system.
- Computers replace mechanical controllers:
  - More accurate.
  - More sophisticated control.
- Engine controllers replace distributor, carburetor, etc.
  - Complex algorithms allow both greater fuel efficiency and lower emissions.

IoT and CPS

- IoT often lower sample rate, larger physical plant.
- CPS often more tightly coupled, higher sample rate.
- Cyber-Physical Systems (CPS) are collections of physical and computer components that are integrated with each other to operate a process safely and efficiently. Examples of CPS include industrial control systems, water systems, robotics systems, smart grid, etc.

Edge computing

- Systems that must respond to the physical world often can’t wait for answers from a remote data center.
- Edge computing:
  - Responsive.
  - Energy efficient.
  - Connected to other devices.

Summary

- Embedded computers are all around us.
  - Many systems have complex embedded hardware and software.
- Embedded systems pose many design challenges: design time, deadlines, power, etc.
- Design methodologies help us manage the design process.